

The Circuitry Guidebook

Cybernetic orchestration, one primitive at a time

Kenan Stipek

Contents

Introduction	1
How to read the examples	2
 I Basics — the machine	 3
1 Prompt	4
1.1 The shape	4
1.2 The minimal prompt	4
1.3 Templates and messages	5
1.4 Typed output	5
1.5 Choosing the model — or not	7
1.6 Prompt-local inputs and assets	7
1.7 Errors, retries, timeouts	7
1.8 What lands in the shadow state	7
1.9 Anti-patterns	8
1.10 See also	8
 2 Dynamic	 9
2.1 The shape	9
2.2 Chain	9
2.3 Tree	10
2.4 The root is a dynamic	11
2.5 Errors inside a container	11
2.6 What lands in the shadow state	12
2.7 Anti-patterns	12
2.8 See also	12
 3 Shadow state	 13
3.1 Three namespaces	13
3.2 Where effects write	14
3.3 Two reading languages, one namespace	14
3.4 Three spellings that fail — and two that fail only at run time	15
3.5 Rendering values	16
3.6 Inside a loop	16
3.7 The runtime namespace	16
3.8 Watching it fill in	17
3.9 Anti-patterns	17
3.10 See also	18
 4 Configuration	 19
4.1 Where config comes from	19

4.2	What a document can set	20
4.3	Trusting a project config	21
4.4	Which model actually runs	22
4.5	Models are opaque strings	22
4.6	Profiles	23
4.7	Preflight and allowlists	23
4.8	Timeouts	24
4.9	Anti-patterns	24
4.10	See also	24
5	Errors	25
5.1	The four knobs	25
5.2	Where failure lands	26
5.3	Designing for degradation	26
5.4	Validation is the first error handler	27
5.5	Dry runs	27
5.6	Anti-patterns	28
5.7	See also	28
II	Cybernetics — the machine steering itself	29
6	If	30
6.1	The shape	30
6.2	Deterministic: mode: cel	30
6.3	Cybernetic: mode: model	31
6.4	Named or not	32
6.5	Same name, both branches	32
6.6	Inside a branch	33
6.7	Branching on structure	33
6.8	Errors on a conditional	34
6.9	Anti-patterns	34
6.10	See also	34
7	Loop	35
7.1	The shape	35
7.2	each — over a collection	36
7.3	while — until the condition says stop	37
7.4	collect	39
7.5	Four read forms, one per question	39
7.6	Named or not	40
7.7	Errors in a loop	41
7.8	Inside nested containers	41
7.9	Anti-patterns	42
7.10	See also	42
8	Reflector	43
8.1	The shape	43
8.2	A reflector in the kitchen	43
8.3	What the plan looks like	44
8.4	Isolation, and what the plan can see	45
8.5	Bounding the feedback	46
8.6	Anti-patterns	46
8.7	See also	46

III The machine in the world	47
9 Composition	48
9.1 use — the composition effect	48
9.2 interface — the contract	50
9.3 Isolation and observation	51
9.4 The library	51
9.5 Composing the dinner	52
9.6 Anti-patterns	53
9.7 See also	53
10 Complexity — scoring and routing	54
10.1 Scoring	54
10.2 Routing	56
10.3 Per-run switches and precedence	58
10.4 Anti-patterns	58
10.5 See also	58
11 Prompt decomposition	59
11.1 Turning it on	59
11.2 What a decomposition does	60
11.3 Bounded at every edge	61
11.4 Reading the plans: --decompose-out	61
11.5 Why the seams matter	62
11.6 Anti-patterns	62
11.7 See also	62
12 Surfaces — CLI, TUI, SDK, MCP, and observability	63
12.1 The CLI: cof	63
12.2 The TUI	65
12.3 The SDK	66
12.4 The MCP server: a Claude session as the model	66
12.5 REST and scheduling	67
12.6 Observability	67
12.7 Anti-patterns	67
12.8 See also	68
13 Tools, adapters, and persistence	69
13.1 Tool effects	69
13.2 Adapters	72
13.3 Runtime plugins	73
13.4 Persistence: the durable side of the shadow state	73
13.5 Anti-patterns	74
13.6 See also	74
14 The whole meal	75
14.1 Reading the run	77
14.2 Turning on the complexity layer	77
14.3 What the meal demonstrates	78
A Grammar	79
A.1 Terminals	79
A.2 Document	79
A.3 Effect — discriminated by type, seven productions	80
A.4 Static semantics (beyond shape)	82

A.5 Cybernetic overlay (config grammar, runtime.complexity)	82
---	----

Introduction

“Control mechanisms that lay their own plans.” — Gordon Pask, *An Approach to Cybernetics* (1961)

This is the long-form companion to the [README](#). The README gets you running and gives you the mental model; the guidebook is the exhaustive tour — every primitive, every state path, every switch, and the methodology that ties them together. It is written as a curriculum: each chapter assumes only the ones before it, and one running example (a dinner party) threads through all of them, so by the last chapter you have seen every part of the language in one document.

The arc has three acts:

I. Basics — the machine. The two base monads the whole language is built from, the shadow state they communicate through, and the config and error handling that keep a run honest.

II. Cybernetics — the machine steering itself. The three effects that read the shadow state and steer: branching, iterating, and planning. This is what the framework is named for.

III. The machine in the world. Orchestrations composing orchestrations, the runtime measuring and routing its own prompts, oversized prompts decomposing themselves, and every surface the outside world arrives through.

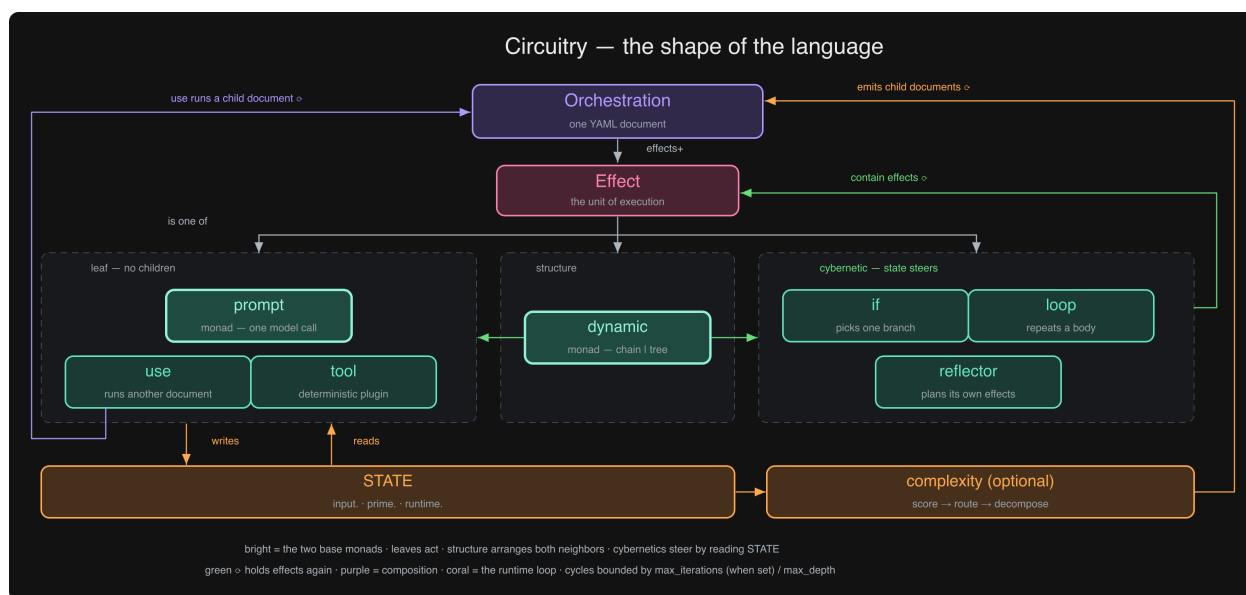


Figure 1: Circuitry — the shape of the language

How to read the examples

Every YAML block in this guidebook is checked by the test suite: positive examples validate with `cof check`, and every anti-pattern is shown under a ✗ and provably fails validation (or, where the validator can only warn, is marked $\triangle$ and provably produces that warning). A handful of anti-patterns are wrong at *run* time rather than validation time — the document is well-formed and passes `cof check`, and the mistake shows only when the run reaches it: most go green with a silently wrong result, and a few fail the run. Those are marked ✗ **runtime**, and the text says exactly what goes wrong.

Two documents sit alongside the guidebook and are referenced from it throughout:

- [Orchestration Reference](#) — the field-by-field table for every effect type. The guidebook explains; the reference enumerates.
- [Grammar](#) — the formal grammar of the orchestration language, derived from `orchestration.schema.json`.

Part I

Basics — the machine

Chapter 1

Prompt

A prompt in Circuitry is not a function call — input in, output out. It is a *computation that returns a value alongside a transformed context*. The value is the model's response. The context is everything around it: which adapter answered, how many tokens it cost, whether it errored, what the rendered prompt actually was. In the vocabulary of the state monad, $f(\text{state}) \rightarrow (\text{state}', y)$ — the prompt is unit, the smallest thing that can be lifted into the run.

That framing is the whole design in miniature. Because a prompt returns *state and value together*, two prompts can be chained without any glue: the second reads what the first wrote. Because the value lands at a path derived from the effect's name, the chaining is deterministic. Everything else in the language — loops, conditionals, planners — is built by reusing this one shape at larger scales.

1.1 The shape

```
Prompt ::= { type: 'prompt', name: NAME,  
             template: TEMPLATE @ messages: Message+,  
             prompt_type?: 'text'|'json'|'object'|'array'|'number'|'boolean',    - default  
               ↪ text  
             schema: JSONSCHEMA,                                             - REQUIRED iff prompt_type ∈ {json,  
               ↪ object, array}  
             model?: STRING, provider?: STRING, provider_fallbacks?: STRING*,  
             params?: MAP, inputs?: MAP,  
             assets?: Asset*, retries?: Retry,  
             timeout_ms?: INT, deterministic?: BOOL,  
             on_error?: 'fail'|'skip'|'continue', description?: STRING }
```

Two fields are required: name and exactly one of template or messages. Everything else is a refinement.

1.2 The minimal prompt

```
- type: prompt  
  name: suggest_dish  
  template: "Suggest one main course for {{input.occasion}}, in one sentence."
```

Run it, and one node appears in state:

```
prime.suggest_dish.value    # the model's answer
prime.suggest_dish.meta    # adapter, model, model_reason, prompt_type, prompt_sent,
                           # tokens_sent, tokens_received, error, fallback_attempts,
                           # created_at, completed_at
```

value is what the prompt is *for*. meta is the transformed context — the second half of the monadic return, and the reason a Circuitry run is a decision record rather than a transcript. prompt_sent in particular is the rendered text the adapter actually received, after every `{{...}}` was interpolated; when a run goes green but the output looks wrong, read it before anything else.

1.3 Templates and messages

template is a [Mustache](#) string. Anything in double braces is looked up in the shadow state: `{{input.occasion}}` reads a caller-supplied value, `{{prime.suggest_dish.value}}` reads an earlier effect's output. Triple-stache `{{{...}}}` skips HTML escaping, which matters when you interpolate code or markup. [Shadow state](#) covers the full addressing rules; the short version is that every reference starts with `input.`, `prime.`, or `runtime.`.

messages is the role-based alternative for models that expect a conversation:

```
- type: prompt
  name: is_vegetarian
  prompt_type: boolean
  messages:
    - role: system
      content: "You are a strict classifier. Reply with only true or false."
    - role: user
      content: "Is this dish vegetarian? {{prime.suggest_dish.value}}"
```

Each content is a template in its own right. The two forms are mutually exclusive — a prompt with both fails validation:

```
## X template and messages are alternatives, never companions
- type: prompt
  name: suggest_dish
  template: "Suggest a main course."
  messages:
    - role: user
      content: "Suggest a main course."
```

1.4 Typed output

prompt_type declares the shape of the value the runtime should extract from the model's reply, and the runtime enforces it. Six types:

prompt_type	value is	Notes
text	string	The default. The reply, as-is.

prompt_type	value is	Notes
boolean	true / false	Lenient: Yes., yes, because ..., **TRUE** , Y parse as true, No./false!/N as false — leading/trailing wrapping (quotes, markdown, punctuation) is stripped first. An answer that still doesn't read as yes/no raises rather than becoming null.
number	number	Lenient: 42, 42., 3.5, -1, 1e3 all parse. A reply with extra words attached (about 42, 42 degrees) raises instead of guessing which number was meant.
json	parsed JSON	schema required.
object	JSON object	schema required.
array	JSON array	schema required.

Structured types require a [JSON Schema](#) (Draft-07), and the value is validated against it before it lands in state:

```
- type: prompt
  name: parse_recipe
  prompt_type: json
  schema:
    type: object
    properties:
      ingredients:
        type: array
        items: {type: string}
      servings: {type: number}
    required: [ingredients]
  template: |
    Extract the ingredient list and the serving count from this recipe.
    Return ONLY a JSON object with "ingredients" and "servings".

    {{input.recipe_text}}
```

Downstream effects then read fields, not text: `{{prime.parse_recipe.value.ingredients}}` in a template, `state.prime.parse_recipe.value.servings > 6` in a CEL expression. This is how effects pass *typed* data to each other, and it is what makes a later loop over the ingredients or an if on the serving count possible at all — a loop cannot iterate prose.

A structured type without a schema is rejected:

```
## X json, object and array all require a schema
- type: prompt
  name: parse_recipe
  prompt_type: json
  template: "Return the ingredients as JSON: {{input.recipe_text}}"
```

Keep schemas small. A schema with more than three or four properties is a smell: the prompt is asking the model to do several things at once, and the fix is several prompts with simpler schemas, each one focused. End every template that expects structure with an explicit in-

struction — “Return ONLY a JSON object with ...”. The extractor is forgiving about code fences and a sentence of preamble, but a reply that wanders produces a null value and a schema failure, and the instruction is what keeps the reply clean.

(A seventh `prompt_type`, `tool`, exists in the schema for tool-call replies and currently passes the reply through untouched. Treat it as reserved.)

1.5 Choosing the model — or not

`model`, `provider`, and `provider_fallbacks` exist on every prompt, and the best-written orchestrations almost never set them. Capability belongs in [configuration](#): a document that hard-wires `model: gpt-4o` cannot run on a local model tomorrow without an edit, and cannot be routed by the [complexity router](#) today. Set `model:` on a prompt only when *that specific step* needs a specific model no matter what the run says — and know that an explicit `model:` outranks even the `--model` flag.

`deterministic: true` asks the adapter for temperature zero (or its equivalent). `params` passes provider-specific generation parameters through untouched — `{temperature: 0.2, top_p: 0.9}` — and, like `model`, is usually left to config.

1.6 Prompt-local inputs and assets

`inputs` supplies template variables scoped to this one call — constants that do not belong in shared state:

```
- type: prompt
  name: pair_wine
  inputs:
    budget: "under forty dollars"
  template: "Pick a wine {{budget}} to pair with: {{prime.suggest_dish.value}}"
```

Prefer shared state for anything another effect might want; use `inputs` for genuinely local constants.

`assets` attaches non-text inputs — `[{kind: image, ref: ./plate.jpg}]` — for multimodal-capable models. `kind` names the asset type; `ref` is a resolvable path or identifier.

1.7 Errors, retries, timeouts

A prompt can fail: the adapter times out, the model returns something that will not parse as the declared type, the provider is down. `retries`, `provider_fallbacks`, `timeout_ms`, and `on_error` govern what happens next, and they are the subject of [Errors](#). The one-line preview: by default a failed prompt fails the run (`on_error: fail`), every attempt is recorded in `meta`, and nothing ever fails silently.

1.8 What lands in the shadow state

<code>prime.<name>.value</code>	# the typed result (null on skip / dry-run)
<code>prime.<name>.meta.adapter</code>	# which adapter answered
<code>prime.<name>.meta.model</code>	# which model — after routing, if routing is on
<code>prime.<name>.meta.model_reason</code>	# "default" "explicit" "router"
<code>prime.<name>.meta.prompt_type</code>	
<code>prime.<name>.meta.prompt_sent</code>	# the rendered prompt, post-interpolation

```

prime.<name>.meta.tokens_sent / tokens_received
prime.<name>.meta.error                # null, or the failure
prime.<name>.meta.fallback_attempts    # the provider attempt chain
prime.<name>.meta.complexity           # present only when scoring is on (chapter 10)
prime.<name>.meta.decomposition        # present only when decomposition fired (chapter
↪ 11)

```

Inside a dynamic the path gains the container's name (`prime.menu.suggest_dish.value`); inside a loop it gains the pass (`prime.courses.iter_0.cook.value`). The rule never changes: the path is derived from the names above it, and nothing else.

1.9 Anti-patterns

One prompt, three jobs. A template that asks the model to *analyze and generate and review* is three prompts wearing one name. Split it — outline → expand → polish — and each step becomes observable, retryable, and routable on its own. Prefer many small effects over few large ones.

Naming an effect after its type. `name: prompt` validates, but the validator warns, and for good reason: generic names are exactly the ones that collide when two of them end up siblings. Name the effect after the job — `suggest_dish`, not `prompt`.

```

## △ validates, but warns: name the job, not the type
- type: prompt
  name: prompt
  template: "Suggest one main course for {{input.occasion}}."

```

A plural name. `summarize_articles` is a signal that one effect is doing many things; the shape you want is a loop over the articles with a singular body effect, `summarize_article`.

Reserved names. `iter_<N>` is reserved for loop passes and cannot be an effect name; `last` is reserved under loops. Names match `^[A-Za-z_][A-Za-z0-9_]*$` and must be unique among siblings.

1.10 See also

- [Orchestration Reference → prompt](#) — the full field table.
- [learn/prompt](#) — text, json, number, and boolean outputs in one runnable file: `cof run learn/prompt`.

Chapter 2

Dynamic

Two prompts raise the question of what the second knows about the first. The answer is one state object every effect writes to and reads from — which is the subject of the next chapter. But before state, there is a smaller question: what is *two prompts*? Not a list. A list has no semantics: it does not say whether the second waits for the first, or whether the two can run at once, or what the pair is called when a third effect wants to read it.

The **dynamic** is the answer: composition as a first-class operator. It takes a list of effects and produces one composite effect with a name, a flow, and a place in state. It is the second base monad — prompt is unit, dynamic is bind made visible — and it is everywhere in the language whether you write it or not. The root of every orchestration is a dynamic. The body of every loop is a dynamic. Each branch of every conditional is a dynamic. A reflector's inner template is a dynamic. Name a composition and you can reuse it; that is the entire trick.

2.1 The shape

```
Dynamic ::= { type: 'dynamic', name: NAME, effects: Effect+,  
              flow?: 'chain' | 'tree',           - default chain  
              max_concurrency?: INT≥1,           - tree only  
              stop_on_error?: BOOL,  
              on_error?: 'fail' | 'skip' | 'continue',  
              labels?: MAP, description?: STRING }
```

name and a non-empty effects list are required. A dynamic *always* contains effects and *never* decides anything — it is pure structure. That is what separates it from the three cybernetic effects in Part II, which also contain effects but exist to read state and steer.

2.2 Chain

```
- type: dynamic  
  name: menu  
  flow: chain  
  effects:  
    - type: prompt  
      name: main_course  
      template: "Suggest a main course for {{input.occasion}}."  
    - type: prompt  
      name: wine
```

```
template: "Pick a wine to pair with: {{prime.menu.main_course.value}}"
```

chain is sequential: each effect runs after the previous one and sees everything it wrote. This is monadic bind, and it is the shape of chain-of-thought — each step conditioned on the last. It is the default flow, so flow: chain can be omitted; writing it is a courtesy to the reader.

Inside the container, the children's paths gain the container's name: wine writes to prime.menu.wine.value, not prime.wine.value. Two spellings reach a sibling from inside the same dynamic — the absolute path, {{prime.menu.main_course.value}}, and the container-relative short form, {{menu.main_course.value}}. Write the absolute one; it is the same spelling a reader outside the container uses, and it never depends on where the template sits.

What does *not* work is the root spelling. main_course lives under menu, so {{prime.main_course.value}} names a node that does not exist. It does not fail — Mustache renders a missing reference as an empty string — which makes this the first of the silent errors this guidebook will keep pointing at:

```
## ✗ runtime – renders empty: main_course lives at prime.menu.main_course, not
↳ prime.main_course
- type: dynamic
  name: menu
  effects:
    - type: prompt
      name: main_course
      template: "Suggest a main course for {{input.occasion}}."
    - type: prompt
      name: wine
      template: "Pick a wine to pair with: {{prime.main_course.value}}"
```

The run goes green, the wine prompt reads “Pick a wine to pair with:”, and the model picks a wine for nothing. prime.menu.wine.meta.prompt_sent is where you would catch it.

2.3 Tree

```
- type: dynamic
  name: courses
  flow: tree
  max_concurrency: 3
  effects:
    - type: prompt
      name: starter
      template: "Suggest a starter to precede {{input.main}}."
    - type: prompt
      name: dessert
      template: "Suggest a dessert to follow {{input.main}}."
    - type: prompt
      name: wine
      template: "Pick a wine for {{input.main}}."
```

tree is parallel: every child launches against the *same* snapshot of state, taken when the dynamic begins. This is the applicative shape — independent computations over shared input — and it is the shape of tree-of-thought: several branches explored at once, joined afterwards. max_concurrency bounds the worker pool; leave it unset and every child runs at once.

The consequence of the shared snapshot is the rule that defines tree: **siblings cannot read each other**. dessert cannot see starter, because when dessert was launched starter had

not written anything yet. A template that tries renders empty, exactly like the wrong-root example above. If one child needs another's output, they are not siblings in a tree — they are steps in a chain.

The two flows compose. A chain whose second step is a tree fans out; a tree whose branches are each a chain runs several pipelines at once; a chain after the tree joins them:

```
- type: dynamic
  name: plan
  flow: chain
  effects:
    - type: prompt
      name: main_course
      template: "Suggest a main course for {{input.occasion}}."
    - type: dynamic
      name: around_it
      flow: tree
      effects:
        - type: prompt
          name: starter
          template: "Suggest a starter to precede {{prime.plan.main_course.value}}."
        - type: prompt
          name: dessert
          template: "Suggest a dessert to follow {{prime.plan.main_course.value}}."
    - type: prompt
      name: menu_card
      template: |
        Write the menu card.
        Starter: {{prime.plan.around_it.starter.value}}
        Main: {{prime.plan.main_course.value}}
        Dessert: {{prime.plan.around_it.dessert.value}}
```

Nesting deepens the path one segment per container: `prime.plan.around_it.starter.value`. The rule is the same at every depth.

2.4 The root is a dynamic

An orchestration's top-level effects: list is the body of an implicit root dynamic, and the document's optional top-level flow: is that dynamic's flow. A document with flow: tree at the top runs all its top-level effects in parallel. The root node is prime itself, which is why a top-level prompt writes to `prime.<name>.value` with no container segment — the container is the root.

```
flow: tree
effects:
  - type: prompt
    name: starter
    template: "Suggest a starter for {{input.occasion}}."
  - type: prompt
    name: dessert
    template: "Suggest a dessert for {{input.occasion}}."
```

2.5 Errors inside a container

A dynamic has its own `on_error`, and one field the leaf effects do not: `stop_on_error`. In a tree, `stop_on_error: true` halts the remaining children when one fails; by default the others

run to completion and the failures are collected. Per-effect `on_error` covers the same intent one effect at a time and is usually the better tool — see [Errors](#).

2.6 What lands in the shadow state

```
prime.<dynamic>.value           # true when the container completed
prime.<dynamic>.meta.flow       # "chain" | "tree"
prime.<dynamic>.meta.error     # null, or the failure (with a breadcrumb into the
  ↪ child)
prime.<dynamic>.<child>.value   # each child, one segment deeper
```

labels — free-form key/value metadata — rides along on the node and into traces; it is observability tagging, not behaviour.

2.7 Anti-patterns

A dynamic that decides. If you find yourself wanting a dynamic to run *some* of its children depending on state, you want an `if`. If you want it to run its children *again*, you want a `loop`. The dynamic’s whole value is that it never surprises you.

Prescribing a fixed number of children. Let the problem dictate the decomposition. A “planning” dynamic with exactly three prompts because three felt right is a chain that will be wrong in both directions.

Tree siblings that read each other. They render empty — see above. When a branch depends on another branch, restructure: chain first, then tree.

Forgetting the container in the path. `{{prime.wine.value}}` for a wine inside menu. Every path is the full path from `prime`.

2.8 See also

- [Orchestration Reference](#) → `dynamic`.
- [learn/dynamic_chain](#) and [learn/dynamic_tree](#).

Chapter 3

Shadow state

Every effect writes to a deterministic path derived from its name, and every later effect reads it there. That single rule is what makes interpolation — and therefore cybernetic feedback — reliable. It is worth being precise about, because the rest of the language rests on it.

The **shadow state** is a real-time graph with the same keys as the orchestration, shadowing the run. The YAML fixes the graph's shape before execution begins — every effect name is a node address — and execution fills it in. A snapshot at any instant is the run's current truth: what has completed, what it produced, which branch was taken, which pass a loop is on, which model answered and why. `--live-state` streams that graph to a file as it grows; `--out` writes the finished one.

3.1 Three namespaces

The shadow state has exactly three root namespaces. Every path starts with one of them.

Namespace	Holds	Written by
input	What the caller supplied	CLI <code>-e / --state</code> , a profile's <code>inputs</code> , a parent's <code>use.inputs</code> , the REST or scheduler payload, the TUI launch form
prime	What effects produced	Every effect, at the path its name and position dictate
runtime	What the framework recorded	The runtime: <code>last_run</code> , <code>effective_settings</code> , <code>plugins</code> , <code>persistence</code> , <code>library_refs</code>

`input` is read-only from the orchestration's point of view; `prime` is where the run happens; `runtime` is the audit trail. Every entry point wraps caller-supplied keys under `input` for you — `cof run dinner.yml -e occasion=anniversary` puts `anniversary` at `input.occasion` — so the spelling inside the document is always `input.<name>`, whether or not the document declares the input in an [interface](#).

3.2 Where effects write

Effect	Writes	Notes
prompt, tool use	prime.<name>.value prime.<name>.value	plus meta a dict of declared outputs — or the child’s whole prime subtree under prime.<name>.<child_effect> when nothing is declared; meta.inputs records what the child received
dynamic named if	prime.<name>.<child>... prime.<name>.<branch_effect>	one segment per container plus the decision under value and meta
unnamed if	into the parent scope	the branch’s effects write as if they were siblings of the if
named loop	prime.<name>.iter_<n>.<step> ...last..., ...collected	plus iteration count and termination reason under value
unnamed loop	into the parent scope	each pass overwrites the same paths
reflector	prime.<name>.inner..., prime.<name>.generated.iter_...	the planning prompt and the plan it ran

The rule beneath the table: **the path is the full path from prime, one segment per named container.** A wine prompt inside a menu dynamic inside a plan dynamic is `prime.plan.menu.wine.value` from anywhere in the document.

3.3 Two reading languages, one namespace

Mustache, inside any template: — a prompt’s template, a tool’s params, a use effect’s inputs, a model-mode condition, an inline orchestration:

```

{{input.occasion}}      caller input
{{prime.suggest_dish.value}} a top-level effect's value
{{prime.menu.wine.value}}  a value inside a dynamic
{{prime.parse_recipe.value.servings}} a field of a structured value
{{prime.courses.last.cook.value}} the final pass of a loop
{{{prime.draft.value}}}} the same read, without HTML escaping

```

CEL, inside any expr: — a cel-mode if, a cel-mode while. Here state is bound to the root, so every path gains that prefix:

```

state.input.guests > 6
state.prime.suggest_dish.value != ""
state.prime.parse_recipe.value.servings >= 8 && size(state.prime.parse_recipe.value.sides)
  ↪ >= 2
!(state.input.diet == 'vegetarian')

```

This is real **CEL**, evaluated by cel-python: comparisons, `&&` `||` `!`, the `?:` ternary, `has()`, the macros (`all`, `exists`, `exists_one`, `map`, `filter`), and the standard functions (`size()`, `int()`, `string()`, `matches()`, ...). Equality is typed the way CEL specifies it, so `1 == true` is false. `cof` check parses every expression, so one that does not parse is an error before the run. state is the only binding; `cof run learn/cel_showcase` runs one branch per construct.

One rule is Circuitry's own. **An expression that reads an unset path is false as a whole**, whatever the operator: a state. path that is missing or resolves through null makes the whole expression false, and the runtime logs a warning that names the path. So `state.prime.fetch_recipe.value != ""` is false after the fetch was skipped, and so is `state.prime.fetch_recipe.value == null`. Only `has()` is exempt: `has(state.input.diet)` asks whether the key is there. `If` covers `strict: true`, which makes an unset path an error.

3.4 Three spellings that fail — and two that fail only at run time

The compiler enforces the namespace rule wherever it can see the path. Three shapes are hard errors from `cof check`, each with the fix spelled out in the message:

```
## X each.in must be rooted at input. / prime. / runtime. (or at an enclosing loop's
↳ binding)
- type: loop
  each: {in: menu, as: course}
  body:
    - type: prompt
      name: cook
      template: "Write the cooking steps for: {{course}}"
```

```
## X in CEL, state.<key> must name one of the three namespaces
- type: if
  if: {mode: cel, expr: "state.diet == 'vegetarian'"}
  then:
    - type: prompt
      name: main_course
      template: "Suggest a vegetarian main course."
```

```
## X a declared interface input is read as {{input.<name>}}, never bare
interface:
  inputs:
    occasion: {type: string, required: true}
effects:
  - type: prompt
    name: suggest_dish
    template: "Suggest one main course for {{occasion}}."
```

Two more shapes are well-formed YAML that the compiler cannot flag. A template is free text and a missing reference is legal Mustache, and the validator checks the paths written after `state.`, not the names written without it. Both pass `cof check`:

```
## X runtime – state. is a CEL binding; in a template it resolves to nothing
- type: prompt
  name: suggest_dish
  template: "Suggest one main course for {{state.input.occasion}}."
```

```
## X runtime – the CEL root is missing, so the first evaluation fails the run
- type: if
  if: {mode: cel, expr: "input.diet == 'vegetarian'"}
  then:
    - type: prompt
      name: main_course
      template: "Suggest a vegetarian main course."
  else:
    - type: prompt
```

```
name: main_course
template: "Suggest a main course."
```

The first goes green and renders “Suggest one main course for .” The second fails the run the first time it is evaluated. `input` is not a name CEL knows (`state` is the only binding), so the `if` records a CEL evaluation error on its `meta.error`, and the run ends with `ok false`. The second failure is loud; the first is silent. For both, read what the run recorded: `meta.prompt_sent` on the prompt, and `meta.error` on the `if`.

3.5 Rendering values

A text value interpolates as itself. A structured value is a dict or a list, and interpolating the whole thing renders its string form — `{'name': 'Roast', 'sides': ['greens', 'bread']}` — which is rarely what a prompt wants. Read fields: `{{prime.dish.value.name}}`, `{{prime.dish.value.sides}}`. To hand a whole collection to the model, prefer iterating it with a `loop`, or ask the producing prompt for prose in the first place.

A node without `.value` is also a legal read and renders the whole node, `meta` included. Nearly always a mistake:

```
## X runtime – renders the whole node, timestamps and all; you meant .value
- type: prompt
  name: card
  template: "Tonight: {{prime.suggest_dish}}"
```

Escaping. `{{...}}` HTML-escapes what it interpolates: `Roast & Co` becomes `Roast & Co`. That is correct for HTML and wrong for a prompt. When the value is prose, code, or anything that may contain `&` `<` `>` `"`, use triple-stache — `{{{prime.draft.value}}}` — and the text passes through untouched.

3.6 Inside a loop

Two bare bindings exist only inside a loop body: `{{<each.as>}}` (the current element of an each loop, `{{item}}` by default) and `{{_loop_index}}` (the zero-based pass number, in each and while loops). They reach every effect in the body however deeply it is nested — inside an `if` branch, a grouping dynamic, an inner loop.

CEL has the same two bindings, one level down: in an `if` condition inside the body, `state.<each.as>` is the current element and `state.iter.index` is the pass number. They are legal only inside the loop that binds them; outside it, `state.course` is the namespace error above. An inner loop’s `each.in` can also start at an outer loop’s binding (`course.steps`), and [Loop](#) shows it.

Within the body, `{{prime.<step>.value}}` means *this pass’s* `<step>`. Resolution is a scope chain: the current iteration first, then the enclosing scope, then root — so a root input or an effect that ran before the loop keeps resolving, and a body step with the same name as an outer effect shadows it for the length of the body. The four read forms for loop state — this pass, a fixed pass, the final pass, every pass — are the subject of [Loop](#); they are the most-consulted table in the reference.

3.7 The runtime namespace

```
runtime.last_run          # run_id, orchestration_path, dry_run, started_at,
↪ completed_at
```

```
runtime.effective_settings      # every resolved setting ...
runtime.effective_settings.sources # ... and which layer supplied each one
runtime.plugins                 # contract version, loaded plugins, hook events
runtime.persistence             # backend, status, loaded_from_persistence
runtime.library_refs            # every ref: a use effect resolved, with its pin
```

`runtime.effective_settings.sources` deserves a sentence of its own. For every setting — model, adapter, out, the complexity switches — it records `cli`, `profile`, `orchestration`, `config`, `default`, or `router`. Every model decision is auditable after the fact, from the state file alone. [Configuration](#) explains the layers.

A document's own runtime: block merges over config: all of it for a file you run by path, only `runtime.complexity` and `runtime.state` for a document that arrives through a library, a fetch or a model ([Configuration](#) has the rule). CEL can read `state.runtime.<key>` and a loop can iterate a `runtime.path`, though there is rarely a reason to.

3.8 Watching it fill in

```
cof run dinner.yml -e occasion=anniversary --live-state ./dinner.live.json
cof run dinner.yml -e occasion=anniversary --out ./dinner.json --pretty
cof run dinner.yml -e occasion=anniversary --json | jq '.prime.suggest_dish'
```

`--live-state` mirrors the graph to the file while the run goes. It writes the first snapshot at once, then at most every half second, and the last write at the end of the run makes the mirror equal to the `--out` state. Each write is atomic, so a viewer pointed at the file watches the graph grow. `--out` is the finished record. `--json` (automatic when stdout is not a terminal) prints it.

A saved shadow state holds each loop's final pass only once. In `--out`, `--json`, and the live mirror, a named loop's last is written as a reference to the pass it aliases, `"last": {"$ref": "iter_2"}`. Every reader that loads a state file links it back: `--state`, a persistence resume, and the TUI's Runs view. A run-time read of `{{prime.courses.last.cook.value}}` does not change. Only a tool that reads the file directly must follow the reference: `jq '.prime.courses | .[.last["$ref"]]'`.

And when a run diverges from what you expected, `inspect_divergence_paths(state)` from the SDK walks the whole tree and returns every node with a `meta.error`, in path order — [Troubleshooting State Paths](#) is the workflow built around it.

3.9 Anti-patterns

Bare keys. `{{occasion}}` for a caller input, each: `{in: menu}`, `state.diet` in CEL. The document has three namespaces; spell them. Undeclared bare template keys are tolerated for backward compatibility, but nothing in this guidebook writes one, and a declared input written bare is an error.

Reading a container as if it were a leaf. `prime.menu.value` is true when the dynamic finished; it is not the wine. `prime.courses.value` is the loop's iteration count and termination reason; it is not the collected courses.

Depending on the string form of a structure. Ask for fields, or iterate.

Forgetting the escaping. One ampersand in a draft, and every downstream prompt reads `&`; Triple-stache prose.

3.10 See also

- [Orchestration Reference](#) → State Path Addressing.
- [Troubleshooting Deterministic State Paths](#).

Chapter 4

Configuration

Capability lives in config, deliberately outside orchestration documents. A document describes *what to do* — the topology, the prompts, the conditions. Config describes *what can do it* — which adapter, which model, which timeout, which persistence backend. Keep the two apart and an orchestration written against a local 8B model this morning runs against a hosted frontier model this afternoon with no edit, and the complexity router in Part III can choose per prompt.

The rule of thumb: an orchestration in the curation library never sets `adapter:` or `model:`. Both fields are legal at the top of a document and on every prompt, and there are moments to use them — a step that must run on a specific model whatever the run says — but each one is a small hard-wiring, and the router steps around it.

4.1 Where config comes from

Resolution is layered, each layer deep-merged over the last:

1. **Sane defaults** — ollama at `http://localhost:11434`, llama3.1:8b.
2. **Global config** — `~/.config/circuitry/config.json`.
3. **Project config** — `circuitry.config.json` or `config.json` in the working directory, once you have trusted it with `cof trust` (see [Trusting a project config](#)).
4. **An explicit file** — `--config <path>` or `CIRCUITRY_CONFIG`, which *replaces* 2 and 3 rather than layering over them.
5. **Environment variables** — `CIRCUITRY_ADAPTER`, `CIRCUITRY_MODEL`, `CIRCUITRY_ADAPTER_URL`, `CIRCUITRY_COMFYUI_URL`, the allowlists `CIRCUITRY_ENABLED_ADAPTERS` / `CIRCUITRY_ENABLED_TOOLS` / `CIRCUITRY_ENABLED_PLUGINS`, and `CIRCUITRY_TRUST_ORCHESTRATION_RUNTIME` (below).

Two exceptions to “the last layer wins”, both about a project config found in the working directory rather than named explicitly. First, `trust`: it applies only once you have run `cof trust` on it (or set `CIRCUITRY_TRUST_PROJECT_CONFIG=1`) — see [Trusting a project config](#). Second, once trusted, it may *narrow* an allowlist the global config set, never widen it: its list is intersected with the global one, and “`enabled_tools`”: `null` in a cloned repo does not re-open what your global config locked down; a warning names what was ignored. A file you name yourself — `--config` or `CIRCUITRY_CONFIG` — skips both checks and is trusted as written.

`cof run` and `cof check` print the config files and environment variables they resolved from on their `Config:` line; `cof doctor` lists them as `Config sources`. A discovered project file appears there even when it was skipped for lack of trust — as (project, not trusted – skipped) rather than (project, trusted) — so the line explains a missing setting as well as a present one.

`cof setup` walks you through creating the global file — it detects local backends and writes a working config. `cof init` writes a project config and a `hello.yml` beside it, and trusts the config it wrote. `cof doctor` tells you what the resolved config can actually reach.

A local-first project config:

```
{
  "default_adapter": "ollama",
  "default_model": "llama3.1:8b",
  "runtime": {
    "adapters": {
      "ollama": {
        "base_url": "http://localhost:11434",
        "timeout_seconds": 600
      },
      "openai": {
        "base_url": "https://api.openai.com/v1",
        "default_model": "gpt-4o-mini"
      },
      "anthropic": {
        "base_url": "https://api.anthropic.com",
        "default_model": "claude-sonnet-4-20250514",
        "max_tokens": 4096
      }
    }
  }
}
```

`default_adapter` and `default_model` are the run defaults. `runtime.adapters.<name>` carries each adapter's own settings — base URL, socket timeout, a per-adapter default model, token limits. API keys are *not* in this file: hosted adapters read them from the environment (`OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, and so on — a `.env` file works), and `cof doctor` names the missing one. Nothing credential-shaped is ever written into serialized state; the redaction layer scrubs it from `runtime.effective_settings` before it lands.

Other top-level keys you will meet later: `runtime.complexity` ([Complexity](#)), `runtime.persistence` and `runtime.plugins.<name>` — per-plugin settings such as MCP servers and a tool's binary ([Tools and persistence](#)), `runtime.library.sources` and `runtime.state.record_children` ([Composition](#)), `plugins` (runtime plugin identifiers), and the three allowlists.

4.2 What a document can set

A document may carry a `runtime:` block and a top-level `plugins:` list of its own. Two runtime settings belong to its author and always apply: `runtime.complexity` and `runtime.state`, each replacing the config block of the same name for that run. Everything else under `runtime` — `adapters`, `plugins`, `persistence`, `library`, `mcp` — is a host setting, and so is a `plugins:` entry config does not already list in `plugins` or `enabled_plugins`. Whether a document's host settings apply depends on how it reached `cof`.

A file you run by path is trusted, the way a script you run is. `cof run ./my.yml`, `cof check` and `cof score` on a path, a local file picked in the TUI Run view, the SDK's `run_orchestration(orchestration_path=...)` and scheduler jobs apply the document's whole `runtime:` block and `plugins:` list. Naming the file is the decision to run it. When it sets host settings, `cof run` prints one line on `stderr` naming them — key paths only, never values — and `cof check` shows the same line:

```
Warning: Applied host settings from my.yml: runtime.adapters.openai.base_url, plugins:
↳ acme.telemetry
```

A file that sets only `runtime.complexity / runtime.state` prints nothing extra. From Python, `run_orchestration(..., trust_document=False)` opts a path out, for a file you did not choose yourself.

A document that arrives any other way is limited to `runtime.complexity` and `runtime.state`: `cof run <name>` from a library source (bundled, folder or github), `cof run-library` and `run_shared_orchestration`, the MCP server's `run_orchestration` and `validate_orchestration` tools, the REST trigger, and plans a model generates. The TUI follows the same rule: its Library view's "run this entry" stays limited whatever the source, and its Chat view's "run it now" stays limited too — the document is still what the model just generated, even once it is saved to disk. Save it, then run that same file with `cof run f.yml` or pick it from the Run view's own local-file list, and it trusts like any other local file. Every limited document's host settings are ignored, and `cof run` (on stderr) and `cof check` print one warning per key saying it must go in `config.json`; an unlisted `plugins: entry` is skipped.

That limit is what makes it reasonable to run a document you did not write. The document chooses what the run does; `config` alone chooses where each adapter sends your prompts and API keys, which binaries and environment a tool gets, which MCP servers start, and where state is written. A composed use: child is held tighter still: it runs on the parent's resolved `runtime` and its own `runtime`: and `plugins`: keys are never read, however the parent was run. Note that `cof fetch` followed by `cof run ./fetched.yml` is running the file by path: read a fetched file before you run it that way, or run it through `cof run-library`.

```
{
  "trust_orchestration_runtime": true
}
```

`trust_orchestration_runtime: true` (or `CIRCUITRY_TRUST_ORCHESTRATION_RUNTIME=1`) trusts every document, however it arrived: its whole `runtime: block` is merged over `config` and every module in its `plugins: list` is imported, with the same one-line notice. Only set it on a host that runs nothing but documents you wrote or have read. With it on, any document the host runs can point an adapter at its own server — which then receives your prompts and the adapter's credentials — change a tool's binary or env, add an MCP server command, redirect persistence, or import arbitrary Python, and use: `ref:` means that includes every document your library sources can reach.

4.3 Trusting a project config

A project config is picked up from whatever directory you run in, and it carries the same authority as your own global file: it can repoint an adapter (and the credentials sent with every prompt), choose the binary and environment a tool plugin executes, start MCP server commands, send run state to a persistence backend, and import runtime plugins. A cloned or downloaded repository can ship one. So, like `direnv allow`, Circuitry applies a discovered project config only after you have said so:

```
cof trust           # the circuitry.config.json / config.json in this directory
cof trust path/to/config.json --yes
cof trust --list    # every trusted file, and whether it still matches
cof untrust        # stop applying it
```

`cof trust` prints every setting the file makes, flags the host-sensitive ones — adapter settings, a tool's binary or env, MCP servers, plugins, persistence, library sources, `trust_orchestration_runtime` — and asks before it records anything (`--yes` skips the question). Trust is kept in `~/.config/circuitry/trusted.json`, beside the global config, as the file's resolved path and the SHA-256 of its contents; your `config.json` is never rewritten. Edit the file and it stops applying until you trust it again.

Until then the file is skipped as a whole, and the run says so once, on stderr:

```
Warning: Skipped project config /home/you/repo/circuitry.config.json: it is not trusted, so
↳ none of its settings apply. Review it, then run `cof trust
↳ /home/you/repo/circuitry.config.json` to apply it.
```

The run itself goes ahead on the other layers, and exit codes do not change. `cof check` prints the same warning; `cof doctor` and the TUI's Doctor and Settings views show the file and its trust state.

Two kinds of file are always trusted, because you named them: the global config, and a file passed with `--config` or `CIRCUITRY_CONFIG`. For CI jobs and containers where the checked-out repository is your own, `CIRCUITRY_TRUST_PROJECT_CONFIG=1` trusts every discovered project config without a trust entry. That is the whole protection switched off: set it only where every directory a run can start in holds a config you would have trusted anyway.

4.4 Which model actually runs

Config sets the default, but it is one voice among several. The full ladder for a prompt's model, most authoritative first:

```
a profile's per-effect model override
> the effect's own model:
  > --model on the command line
    > a profile's run-level model:
      > a profile's per-effect routing pin
        > the complexity router
          > the document's top-level model:
            > CIRCUITRY_MODEL
              > project config default_model
                > global config default_model
                  > the built-in default
```

Everything above the router is a model a human named on purpose, and the router never overrules one. Everything below it is a default the router exists to replace. Two consequences worth stating: an effect's own model: beats `--model`, because `--model` sets the *run default* and a step that names its own model has opted out of the run default whoever supplied it; and a profile is how you retarget one step from outside the document without editing it.

Adapters resolve the same way minus the router: `--adapter > profile > document adapter:`
`> CIRCUITRY_ADAPTER > config.`

The less usual part is that every one of these decisions is recorded. `runtime.effective_settings.sources` names the layer that supplied each setting — `cli`, `profile`, `orchestration`, `config`, `default`, or `router` — and each prompt's `meta.model_reason` says `explicit`, `default`, or `router`. You can reconstruct why a model ran from the state file alone, months later. `cof info <orchestration>` shows the resolved complexity settings and their provenance before you run anything.

4.5 Models are opaque strings

The runtime does not interpret model names. For most adapters they are provider model identifiers — `llama3.1:8b`, `gpt-4o-mini`. For a broker they can be something else entirely: the [CyberDiner](#) adapter is a job-queue LLM network, and there `model:` names a *capability tier* — `cheap`, `fast`, `good`, `good-fast`, `alpha` — that the network resolves to whatever is serving that tier. The adapter hides `submit-and-poll` behind the same synchronous `generate()` every

other adapter implements, so the document neither knows nor cares. `cof list --models <adapter>` asks an adapter what it can serve.

Twenty-nine adapters ship in-tree behind one Adapter protocol — hosted APIs, self-hosted servers (vllm, llama.cpp, LM Studio, TGI), aggregator routes (OpenRouter, LiteLLM), the CyberDiner broker, and `host_claude`, the adapter that turns a Claude session into the model over MCP ([Surfaces](#)). `cof list --extensions` prints the compiled-in set. [Tools and persistence](#) covers writing one.

4.6 Profiles

A profile is a YAML file that overlays one run — defaults, inputs, per-effect overrides, persistence — without touching the orchestration:

```
## profiles/fast.yml
adapter: ollama
model: llama3.2
out: runs/fast.json
inputs:
  occasion: "weeknight"           # merged under input.*; -e still wins
effects:                          # keyed by effect path, as in state minus the prime. root
  plan_courses:
    model: cheap
    provider: cyberdiner
  season:
    enabled: false                # switch an effect off for this run
  replan_service:
    routing: premium              # pin to a named routing band (chapter 10)
persistence:
  backend: jsonl-file
  path: runs.jsonl
```

```
cof run dinner.yml --profile fast
```

Profiles are discovered at `<orchestration_dir>/profiles/<name>.yaml` first, then `<cwd>/profiles/<name>.yaml` and validated against a schema — an unknown effect path fails with the list of valid ones. `enabled: false` is the flagship: it turns a reflector’s agentic planning off for one run and leaves everything else intact, writing a skip node in its place so downstream paths still resolve (to null). A container disabled this way disables its whole subtree; a condition (`if`: on an `if`, `while`: on a loop) cannot be disabled on its own — disable the container.

The recorded profile is enough to reproduce the run without the file: `cof run dinner.yml --profile-from-state ./runs/fast.json` reconstructs it from `runtime.effective_settings.profile`. Redacted secrets are the exception — reconstruction refuses to replay a `***REDACTED***` sentinel and tells you to bring the file.

[Named Profiles](#) has the full precedence rules and the persistence table.

4.7 Preflight and allowlists

Every adapter, tool plugin, and runtime plugin implements a `check()`. Before the first model call, `cof run` runs the checks for every extension the orchestration references — the adapter is reachable, the plugin’s binary is installed, the API key is set — and refuses to start a run that would fail halfway. `cof doctor` runs every check and reports; `cof doctor --generate` also makes a live model call. `--skip-preflight` bypasses the gate when you know better.

An adapter that only optional steps use does not block the run. When every prompt on an adapter has `on_error: skip` or `continue`, a missing credential for it is a warning, not a failure, and those steps skip. [Errors](#) has the exact rule.

The allowlists are the other gate: `enabled_adapters`, `enabled_tools`, and `enabled_plugins` in config (or their `CIRCUITRY_ENABLED_*` environment forms) restrict which extensions a run may touch. Unset means everything compiled in is available; set means an orchestration referencing anything else fails at validation, before any call. `cof check` follows the document's use children — `path:`, `ref:` and plain `inline:` — and applies the same lists to each. At run time every adapter and tool is checked again as it is built, which covers what no document text shows: a templated `inline: child` once it renders, reflector and decomposition plans, `--adapter`, the config's `default_adapter`, and a profile's provider overrides. `cof run-library --service-profile` keeps the lists too. In a deployment that should never shell out, `enabled_tools` is where you say so. [Threat Model](#) covers the reasoning.

4.8 Timeouts

Two clocks. `timeout_ms` on an effect bounds that effect. `runtime.adapters.<name>.timeout_seconds` in config bounds the adapter's socket — and a large local model can need cold-load headroom well beyond any single effect's budget, which is why the example above gives ollama ten minutes. The two compose: the effect's budget is the one you tune per step; the adapter's is the one you set once per machine. [Errors](#) is next.

4.9 Anti-patterns

Hard-wiring the provider. `adapter: openai` at the top of a document that is meant to be shared. Put it in config, or in a profile.

Credentials in YAML. Never. Environment or config; the run record redacts config, and nothing redacts a template.

Reaching for `--model` to retarget one step. `--model` sets the run default; a per-effect `model:` in the document beats it. Use a profile's `effects: block` to retarget a single step from outside.

Trusting PATH. The dev tooling is pinned in `requirements-dev.txt` so that green locally means green in CI. The same discipline applies to models: pin the model in config, not in memory.

4.10 See also

- [Named Profiles](#).
- [Library Sources](#) — `runtime.library.sources`.
- [CyberDiner demo runbook](#) — a broker adapter end to end.

Chapter 5

Errors

Runs degrade deliberately, never mysteriously. A model call can time out, return something that will not parse, or come back from a provider that is down; a tool's binary can be missing; a child orchestration can fail validation. Circuitry's position on all of it is the same: the failure is recorded where it happened, the policy for what happens next is declared on the effect, and the run record says not just *that* something failed but what the framework did about it.

This chapter comes before the cybernetic effects on purpose. A control loop that steers on state has to know what a failed step leaves in state, and the answer is always the same shape.

5.1 The four knobs

```
- type: prompt
  name: pair_wine
  template: "Pick a wine to pair with: {{prime.suggest_dish.value}}"
  retries: {max_attempts: 3, backoff_ms: 500}
  provider_fallbacks: [ollama, "openai:gpt-4o-mini"]
  timeout_ms: 60000
  on_error: skip           # no sommelier tonight – dinner goes on
```

retries — {max_attempts, backoff_ms} on a prompt. Each attempt re-renders nothing and re-sends the same prompt; backoff_ms is the pause between attempts. Each attempt is recorded, and meta.retries_used says how many it took. Prompts only — a tool that failed is not usually improved by asking again, and a use child carries its own policies.

provider_fallbacks — an ordered list of providers to try when the primary errors. Each entry is an adapter[:model] token: ollama means the ollama adapter with the run's default model; openai:gpt-4o-mini names both. The attempt chain — every adapter and model tried, in order, with its outcome — lands at meta.fallback_attempts, and meta.fallback_recovered is true when it took more than one. You can always see who actually answered. (An effect's own provider: sets the *primary* the same way; both are usually run policy, set in config or a profile, rather than something a document author writes.)

timeout_ms — the effect's budget. Separate from the adapter's socket timeout in config (runtime.adapters.<name>.timeout_seconds), which is per machine rather than per step; a large local model needs cold-load headroom there that no single effect should have to carry.

on_error — what happens when the attempts are exhausted. Three values on every effect except loops:

on_error	The effect	The run
fail (default)	records the error	stops; the failure propagates to the root with a breadcrumb path
skip	records the error, writes value: null	continues with the next effect
continue	records the error, value stays null	continues with the next effect

For a leaf effect — prompt, tool, use — skip and continue are the same degradation: a null value, an error in meta, and a run that keeps going. Downstream templates render the missing value as empty; a downstream CEL expression that reads it is false as a whole, whatever its operator. The difference shows on an if: skip runs *neither* branch (branch: null), while continue takes the else branch as a degraded default. Choose by what downstream needs.

Loops have their own vocabulary — fail / break / continue — because a failed *pass* is a different thing from a failed effect. [Loop](#) covers it; the short form is that break ends the loop at the failed pass and continue drops that pass and goes on, and neither a dropped pass nor a broken one counts toward last.

5.2 Where failure lands

```
prime.<name>.value           # null after a skip
prime.<name>.meta.error      # the message
prime.<name>.meta.fallback_attempts # [{adapter, model, status, error}, ...]
prime.<name>.meta.fallback_recovered # true when a fallback answered
prime.<name>.meta.retries_used  # present when a retry succeeded
prime.<container>.meta.error   # "prime.menu.wine: ..." — a breadcrumb into the child
```

A container that fails closes its own node with value: false and a meta.error that names the child path, so from the root down you can follow the breadcrumbs to the leaf. inspect_divergence_paths(state) in the SDK does the walk for you and returns every errored node in path order.

Two things never happen. A failure never leaves a *partial* value: a use child whose run failed discards its scratch state, and nothing lands at the parent's value. (With runtime.state.record_children: true, the child's partial record is kept beside that value for the audit trail. It is a record, not something downstream reads.) And a failure never goes unrecorded: even on_error: skip writes the error into meta, and the per-effect on_effect_complete hook fires for the failed node as it does for a successful one, carrying the error. That includes a failure the child absorbed: when an effect inside a use child skipped its own failure, the use node lists it in meta.child_errors as {path, error}, so a composed run that went green still says what it lost.

5.3 Designing for degradation

The dinner party has a step that fetches a recipe from the web for inspiration. The web is optional; dinner is not:

```
- type: tool
  name: fetch_recipe
  provider: web_fetch
  params: {url: "{{input.recipe_url}}"}
}
```

```

on_error: skip

- type: prompt
  name: plan_courses
  template: |
    Plan three courses for {{input.occasion}}.
    Inspiration, if any: {{prime.fetch_recipe.value}}

```

When the fetch fails, `plan_courses` reads “Inspiration, if any:” and carries on. The model gets less; the run gets a menu; the state file says the fetch failed and why. That is the pattern: put skip on the effects whose absence downstream can tolerate, leave fail on the ones it cannot, and write the downstream templates so that an empty interpolation reads as “none” rather than as a broken sentence.

The other half of the pattern is *not* to reach for `on_error` when what you want is a decision. If the missing recipe should change what happens next — a different planning prompt, say — that is an `if`. Remember the CEL rule from [Shadow state](#): an expression that reads a null or missing path is false as a whole. So test positively for the value you need — `size(state.prime.fetch_recipe.value) > 0` — and let the missing case fall to else. To branch on the failure itself, test the error: `state.prime.fetch_recipe.meta.error != null` is true when the fetch failed, and false when it succeeded, because a null error makes the whole expression false. Do not write `state.prime.fetch_recipe.value == null`: it reads a null path, so it is false even after the skip.

5.4 Validation is the first error handler

Most failures should never reach the runtime. `cof check` validates a document against the schema and the static rules — namespace-rooted paths, unique sibling names, required schemas, use cycles, unfetched library sources — and then runs *preflight*: every adapter and plugin the document references reports whether it can run (`check()`), so a missing API key or an uninstalled binary stops the run before the first model call rather than after the tenth. `cof doctor` runs the same checks against everything compiled in. `--skip-preflight` exists for the moments you know better.

Preflight reads `on_error` too. When every prompt that uses an adapter has its own `on_error`: skip or continue, that adapter is a *soft* dependency: a missing credential becomes a warning that names the effects that will skip, `cof check` passes, and the run goes on with those values null. One prompt on that adapter without the handling makes it a hard dependency again, and the error names that prompt. Only each prompt’s own `on_error` counts, not an enclosing container’s. An adapter name that does not exist is always a hard failure.

Warnings are advisory: deprecated spellings and type-keyword names are reported and never change the exit code.

5.5 Dry runs

`cof run ... --dry-run` executes the whole orchestration without calling a model: every prompt writes `value: null` and a meta block with the rendered `prompt_sent`; CEL conditions evaluate for real, while model-mode ones take a fixed answer (an `if` takes then, a `while` stops after its minimum passes); each loops iterate whatever collection is already in input; tools are skipped. It is the fastest way to see the state graph a document will produce, and to read every rendered prompt before spending a token. The [complexity scorer](#) runs in a dry run too.

5.6 Anti-patterns

skip everywhere. A run where every effect is skippable cannot fail, and therefore cannot tell you anything. Skip the optional; fail the essential.

Retrying a parse failure without changing the ask. If a json prompt fails its schema three times, the fourth attempt will too. The fix is the template — “Return ONLY ...”, a smaller schema — not `max_attempts`.

Testing for null in CEL. `state.prime.x.value == null` is false after a skip, because an expression that reads a null path is false as a whole. Test positively for the value, or test `meta.error != null`.

Timeouts in the wrong clock. A cold-loading 70B model needs minutes; put that in the adapter’s `timeout_seconds`, not on every effect.

5.7 See also

- [Orchestration Reference](#) → `prompt` — `retries`, `provider_fallbacks`, `timeout_ms`, `on_error`.
- [Troubleshooting Deterministic State Paths](#).

Part II

Cybernetics — the machine steering itself

Chapter 6

If

“People can even have conversations with themselves.” — Gordon Pask, *Learning Strategies and Learning Styles* (1988)

Everything in Part I is *open-loop*: a static directed graph, every decision made at authoring time. A long chain or a wide tree can run a hundred prompts and still cannot surprise itself, because nothing it produces changes what it does next. What turns it into a *closed-loop* control system is two ideas — **branching on observed state** and **iterating on observed state** — and this chapter is the first.

An if is a predicate over state plus two continuations. The interesting case is the model-evaluated predicate: the LLM reads what the run has produced and answers a yes/no question about it. The model becomes the sensor, not just the actuator. A monitor compares, a controller adjusts — the orchestration has begun to observe itself running.

6.1 The shape

```
If      ::= { type: 'if', if: Condition, then: Effect*,
              else?: Effect*, name?: NAME,
              threshold?: NUMBERS[0,1],           - recorded with model-mode
              ↪ decisions
              on_error?: 'fail'|'skip'|'continue',
              labels?: MAP, description?: STRING }
```



```
Condition ::= { mode: 'model', template: TEMPLATE } - the LLM answers yes/no
              | { mode: 'cel',   expr: CEL,          - deterministic
                  strict?: BOOL }                  - default false: an unset path is
              ↪ false
```

if and then are required. name is optional, and whether you give one changes where the branch writes — below.

6.2 Deterministic: mode: cel

```
- type: if
  name: check_diet
  if:
    mode: cel
    expr: "state.input.diet == 'vegetarian'"
```

```

then:
- type: prompt
  name: main_course
  template: "Suggest a vegetarian main course for {{input.occasion}}."
else:
- type: prompt
  name: main_course
  template: "Suggest a main course for {{input.occasion}}."

```

The expression is evaluated against the shadow state, with state bound to its root, in the CEL described in [Shadow state](#): comparisons, && || !, ?:, has(), the list macros, and the standard functions. It is deterministic — the same state always takes the same branch — and free. Reach for it whenever the decision is already sitting in state as data: a caller input, a field of a structured prompt output, a loop’s termination reason, a complexity score.

An expression that reads an unset path is false, not an error, and the runtime logs a warning that names the path. That is convenient for missing paths and treacherous for typos: `state.input.deit == 'vegetarian'` quietly takes else forever. Read the branch recorded on the node when a decision surprises you. When a missing field must not pick a branch — a safety gate, a rule that ends an order — add `strict: true` beside `mode: cel`. Then an unset path is an error on the if, handled by its `on_error`.

An expression that cannot be evaluated at all is always an error. A name CEL does not know, or a function called on the wrong type, is recorded on the node’s `meta.error`, and under the default `on_error: fail` the run fails. The branch is never chosen silently.

6.3 Cybernetic: mode: model

```

- type: if
  name: quality_gate
  if:
    mode: model
    template: "Is this menu balanced and ready to serve?\n\n{{prime.plan_courses.value}}"
  then:
    - type: prompt
      name: verdict
      template: "Approve this menu in one line: {{prime.plan_courses.value}}"
  else:
    - type: prompt
      name: verdict
      template: "Rewrite this menu so it is balanced: {{prime.plan_courses.value}}"

```

The runtime renders the template, wraps it — *“Evaluate the following condition and respond with ONLY ‘yes’ or ‘no’: ... Answer (yes/no):”* — and parses the reply leniently: Yes., yes, because ..., ****TRUE**** and Y are true, No./false!/N are false — a real model rarely answers with a bare token, so surrounding quotes, markdown emphasis and trailing punctuation are stripped before the answer is read. A reply that still doesn’t parse as yes/no is not read as false — it **raises**, so `on_error` handles it exactly like a failed model call would; taking else for an unreadable answer is only ever the explicit `on_error: continue` opt-in. So phrase the template as a question with a yes/no answer, and put the evidence in it; the model sees only what the template shows.

This is the sensor. The predicate is a judgement no CEL expression could make — *is this good enough?* — and the model makes it against the run’s own output. The cost is one model call and the nondeterminism that comes with it; the return is a control loop that can evaluate quality, relevance, safety, completeness.

threshold (default 0.5) is recorded alongside the decision in meta. The built-in evaluator asks for a categorical yes/no, so today the threshold is metadata rather than a cut on a probability; it exists so that an evaluator that returns confidence can honour it without a schema change.

6.4 Named or not

A **named** if is a container: its branch effects nest under it — `prime.check_diet.main_course.value` — and the node records the decision:

```
prime.check_diet.value.result      # true / false
prime.check_diet.value.branch     # "then" / "else"
prime.check_diet.value.effects    # what ran
prime.check_diet.meta.mode        # "cel" / "model"
prime.check_diet.meta.condition_result
prime.check_diet.meta.threshold
prime.check_diet.meta.answer      # mode: model only – the raw reply
prime.check_diet.meta.adapter     # mode: model only
prime.check_diet.meta.model       # mode: model only
prime.check_diet.main_course.value # the branch effect's output
```

An **unnamed** if is transparent: it contributes no path segment, and its branch effects write into the enclosing scope as though they were siblings of the if itself — `prime.main_course.value`. No decision record is kept. Use the unnamed form when the branching is an implementation detail and downstream just wants `main_course`; use the named form when the decision itself is something you will inspect, branch on again, or cite.

6.5 Same name, both branches

Whichever way dinner went, downstream wants *the main course*. Give the effect the same name in both branches, and every later template reads one path:

```
- type: prompt
  name: wine
  template: "Pick a wine to pair with: {{prime.check_diet.main_course.value}}"
```

This is the single most important habit in the chapter. Two branches with different inner names — `veg_main` and `meat_main` — force every downstream effect to know which branch ran, which is exactly the coupling the if was supposed to absorb:

```
## X runtime – downstream cannot address "the main course": one of these two paths is
↳ always empty
- type: if
  name: check_diet
  if: {mode: cel, expr: "state.input.diet == 'vegetarian'"}
  then:
    - type: prompt
      name: veg_main
      template: "Suggest a vegetarian main course for {{input.occasion}}."
  else:
    - type: prompt
      name: meat_main
      template: "Suggest a main course for {{input.occasion}}."
- type: prompt
  name: wine
  template: "Pick a wine for {{prime.check_diet.veg_main.value}}
↳ {{prime.check_diet.meat_main.value}}"
```

6.6 Inside a branch

A branch is a sequence, and a step in it reads the steps before it. `{{prime.<step>.value}}` inside a branch resolves the branch's own earlier steps first, in templates and in CEL, in a nested if, and in an if inside a loop body. A named if layers its branch over the enclosing scope the way a loop body does, so a branch step shadows a root step with the same name:

```
- type: prompt
  name: main_course
  template: "Suggest a main course for {{input.occasion}}."
- type: if
  name: check_diet
  if: {mode: cel, expr: "state.input.diet == 'vegetarian'"}
  then:
    - type: prompt
      name: main_course
      template: "Suggest a vegetarian main course for {{input.occasion}}."
    - type: prompt
      name: wine
      template: "Pick a wine for {{prime.main_course.value}}."
```

Inside the then branch, `{{prime.main_course.value}}` is the vegetarian main course. After the if, the same spelling is the root `main_course` again, and the branch's is `prime.check_diet.main_course`. The full path works in both places. That is the reason to prefer it: it does not depend on where the template sits.

6.7 Branching on structure

The decisive pattern in Part I was *typed output feeds typed reads*. It pays off here: a json prompt produces a field, and a CEL if routes on it.

```
- type: prompt
  name: classify_request
  prompt_type: json
  schema:
    type: object
    properties:
      kind: {type: string}
      confidence: {type: number}
    required: [kind]
  template: |
    Classify this dinner request as "simple" or "elaborate".
    Return ONLY a JSON object with "kind" and "confidence".

    {{input.request}}

- type: if
  name: route
  if:
    mode: cel
    expr: "state.prime.classify_request.value.kind == 'elaborate' &&
    ↪ state.prime.classify_request.value.confidence >= 0.7"
  then:
    - type: prompt
      name: plan
      template: "Plan a five-course tasting menu for: {{input.request}}."
  else:
    - type: prompt
```

```
name: plan
template: "Plan a simple three-course dinner for: {{input.request}}"
```

Classify-then-route is a bundled pattern (patterns/classify_then_route) and a bundled utility (utilities/route) — [Composition](#) shows how to call them rather than restate them.

6.8 Errors on a conditional

on_error applies to the *condition*: a model call that fails, a CEL expression that cannot be evaluated, or a strict expression that reads an unset path. fail propagates. skip runs neither branch and records branch: null. continue takes else as a degraded default — the branch you should therefore write as the safe one. A failure *inside* a branch is the branch effect's own business, under its own on_error.

6.9 Anti-patterns

Different names in the two branches. Above. Same name, always.

A CEL expression without state. — input.diet == 'vegetarian' — is legal YAML, passes validation, and fails the run on its first evaluation. Marked in [Shadow state](#).

A model condition with no evidence. “Is the menu good?” with nothing interpolated asks the model to judge something it cannot see. Put the value in the template.

Testing for null. state.prime.fetch_recipe.value == null is false when the fetch was skipped, because an expression that reads a null path is false as a whole. Test positively for what you need, or test state.prime.fetch_recipe.meta.error != null.

Nesting if three deep to express a routing table. Classify once into a kind, then branch once on it — or use utilities/route and read the chosen key.

6.10 See also

- [Orchestration Reference](#) → if.
- [learn/conditional](#) · [patterns/classify_then_route](#).

Chapter 7

Loop

The loop body executes, writes state, and the continuation condition evaluates against that new state — each iteration’s output is the next iteration’s input. This is feedback in the literal cybernetic sense: the output of the system is fed back as input, and the system decides from what it observes whether to go around again.

Two kinds of loop answer two kinds of question. `each` asks “for every one of these?” and walks a collection. `while` asks “again?” and consults a condition — a CEL expression, or the model tasting the dish. Neither is capped unless you say so. An `each` loop walks a collection resolved before its first pass, so it can run long but never forever. A `while` loop runs until its condition says stop, and one whose condition never does runs indefinitely. `max_iterations` is the ceiling you set over that runaway feedback. It is opt-in, and when you set it, it must be at least 1: `cof check rejects max_iterations: 0`.

7.1 The shape

```
Loop ::= { type: 'loop', body: Effect+,  
  each: { in: STATE_PATH | BINDING_PATH,  
    ↪ at an outer loop's as  
      as?: NAME,  
      truncate?: BOOL }  
    ↪ collection fails the loop  
  ⊕ while: Condition,  
    ↪ always sequential  
  name?: NAME,  
    ↪ collected nodes  
  collect?: NAME,  
  flow?: 'chain' | 'tree', max_concurrency?: INT≥1,  
  max_iterations?: INT≥1,  
    ↪ may not exceed it  
  min_iterations?: INT,  
  on_error?: 'fail'|'break'|'continue',  
  labels?: MAP, description?: STRING }
```

- resolves to an array; may start
- defaults to item
- default false: too long a
- checked before each pass;
- named ⇒ `iter_<N>` / last /
- a body step; needs a named loop
- each loops only
- no default; an each collection

body and exactly one of each / while are required.

7.2 each — over a collection

```
- type: loop
  name: courses
  each:
    in: prime.menu.plan_courses.value
    as: course
  collect: cook
  body:
    - type: prompt
      name: cook
      template: "Write the cooking steps for: {{course}}"
```

`each.in` is a root-relative state path that must resolve to an array at run time — `input.menu` for a caller-supplied list, `prime.menu.plan_courses.value` for the output of an array/json prompt, or a `runtime.path`. `as` names the current element for the body (item by default). The body runs once per element, in order.

The path is checked statically for its root — a bare key or a state. spelling is a hard error from `cof check`, with the fix named — and dynamically for its type. A path that resolves to nothing, or to something that is not a list, ends the loop with termination reason `collection_unresolved` and zero passes. Never silent success: a text prompt that was supposed to produce a list, and produced prose, is caught here.

```
## X runtime — plan_courses is text; each.in needs an array, so the loop terminates
↳ collection_unresolved
- type: prompt
  name: plan_courses
  template: "List three courses for {{input.occasion}}."
- type: loop
  name: courses
  each: {in: prime.plan_courses.value, as: course}
  body:
    - type: prompt
      name: cook
      template: "Write the cooking steps for: {{course}}"
```

The fix is upstream: `prompt_type: array` with a schema, so the producing prompt is held to the shape the loop needs.

The collection is the bound. An each loop knows its length before the first pass, so hitting `max_iterations` is never a runaway; it means the list is longer than you said it could be. Without `max_iterations`, the loop runs every element, and nothing below applies. With it, a collection with more elements than the cap fails the loop at start, with no pass run. The error names both numbers and the `each.in` path: *each loop 'courses' (prime.menu.plan_courses.value): collection has 12 items but max_iterations is 8 —* Raise the cap, bound the collection upstream, or say that the first *N* are enough:

```
- type: loop
  name: courses
  max_iterations: 8
  each: {in: prime.menu.plan_courses.value, as: course, truncate: true}
  body:
    - type: prompt
      name: cook
      template: "Write the cooking steps for: {{course}}"
```

With `truncate: true` the loop cooks the first eight courses and records why it stopped: `ter-`

mination reason `max_iterations_reached`, plus `termination.unvisited`, the count of courses it never reached. A shorter menu is not truncated; the list can end before the cap.

In parallel. An each loop is a chain by default. `flow: tree` runs every pass concurrently — the loop’s *iterations* fan out, while the steps inside one pass still run in order — and `max_concurrency` bounds the pool. Results are assembled in the original order whatever order they finished in:

```
- type: loop
  name: courses
  flow: tree
  max_concurrency: 4
  collect: cook
  each: {in: prime.menu.plan_courses.value, as: course}
  body:
    - type: prompt
      name: cook
      template: "Write the cooking steps for: {{course}}"
```

7.3 while — until the condition says stop

```
- type: loop
  name: season
  while:
    mode: model
    template: "Taste this. Does the seasoning need adjusting?\n\n{{prime.adjust.value}}"
  min_iterations: 1
  max_iterations: 5
  body:
    - type: prompt
      name: adjust
      template: "Adjust the seasoning of {{prime.check_diet.main_course.value}} and
    ↪ describe the dish now."
```

The condition is evaluated *before* each pass, and it sees the pass that just finished under the same within-iteration names the body uses — `{{prime.adjust.value}}` in the condition is the latest adjustment. Before the first pass there is nothing to see yet, and the name falls through to the enclosing scope (empty, here). `min_iterations` forces that many passes regardless of the answer — the way to say “always taste at least once” — and `max_iterations` stops the loop whatever the model thinks. It has no default: without it, a while loop runs until its condition says stop, however many passes that takes. A loop stopped that way completes normally, but it is not recorded as converged: its termination reason is `max_iterations_reached`, and a `--verbose` run prints a warning line. A while loop is always sequential; `flow` does not apply.

Model mode wraps the template the way `if` does — “... *Should the loop continue? Answer (yes/no):*” — and parses the reply just as leniently (see [If](#)): `Yes./yes`, because `.../TRUE**` continue, `No./false!` stop, and an answer that doesn’t parse as either raises instead of silently stopping the loop. Each check’s raw reply, adapter and model land on `meta.answer/meta.adapter/meta.model`. So phrase it as a question whose *yes* means “go around again”. CEL mode is deterministic, and reads whatever the previous pass left in state:

```
- type: prompt
  name: guest_count
  prompt_type: number
  template: "How many guests are coming to {{input.occasion}}? Reply with a number only."
```

```
- type: loop
  name: courses_until_full
  while:
    mode: cel
    expr: "state.prime.guest_count.value > 6"
  max_iterations: 3
  collect: course
  body:
    - type: prompt
      name: course
      template: "Suggest one more course for {{input.occasion}}, pass {{_loop_index}}."
```

One thing a CEL condition cannot do is read the loop's own collected or last — both are written when the loop *completes*, so from inside the loop the path is missing, and an expression that reads a missing path is false. A while written that way runs zero passes and terminates condition_false. *Where* you read loop state matters as much as *what* you read; the next section is the map.

7.3.1 Refining across passes

The condition sees the previous pass. The *body* of a named loop does not: inside the body, `{{prime.adjust.value}}` means *this pass's* adjust, which has not been written yet when the pass begins, so it renders empty. `last` and `iter_<N>` are post-loop spellings. There is no spelling in a named loop for “the pass before this one”.

When the body needs to build on its own previous output — taste, adjust, taste again — use an **unnamed** loop and let the body overwrite the same name it reads:

```
- type: prompt
  name: dish
  template: "Describe {{prime.check_diet.main_course.value}} as first plated, seasoning
↳ included."
- type: loop
  while:
    mode: model
    template: "Taste this. Does the seasoning still need
↳ adjusting?\n\n{{prime.dish.value}}"
  max_iterations: 5
  body:
    - type: prompt
      name: dish
      template: "Adjust the seasoning and describe the dish now:\n\n{{prime.dish.value}}"
- type: prompt
  name: card
  template: "Write the menu line for: {{prime.dish.value}}"
```

An unnamed loop writes into the enclosing scope, so each pass's dish replaces the last: the body reads the previous pass, the condition tastes the latest, and after the loop `prime.dish.value` is the final version with no last to spell. The seed prompt before the loop gives the first pass something to read. What you give up is history — no `iter_<N>`, no collected — which is the right trade when only the final state matters. Keep the loop named when the record of every pass is the point.

7.4 collect

collect: <step> gathers one body step's value from every pass into an array at `prime.<loop>.collected.value` in pass order. It needs a named loop — the array has to live somewhere — and it is the usual way a loop hands its work to the effect after it:

```
- type: prompt
  name: menu_card
  template: "Write the menu card from these courses: {{prime.courses.collected.value}}"
```

A collect target that a profile switched off contributes no slot, and nothing after a pass that broke the loop is collected. A pass dropped under `on_error: continue` is, today, the exception; [Errors in a loop](#) says what it leaves.

7.5 Four read forms, one per question

This is the table to keep. Four *different* questions get four *different* paths, and substituting one for another does not fail — it renders something plausible and wrong.

You want	Write	Legal where
A step's output in the current pass	<code>{{prime.<step>.value}}</code>	inside the body, and inside the while condition
One specific past pass	<code>{{prime.<loop>.iter_<N>.<step>.value}}</code>	after the loop only
The final completed pass	<code>{{prime.<loop>.last.<step>.value}}</code>	after the loop only
Every pass's output of one step	<code>{{prime.<loop>.collected.value}}</code>	after the loop (requires collect)

The rules of the form:

- **Resolution is a scope chain.** Inside the body, `prime.<step>` means the current iteration first, then the enclosing scope, then root. A root input or an effect that ran before the loop keeps resolving; a body step named the same as an outer effect shadows it, inside the body only.
- **last is the last pass that completed.** A pass that errored under `on_error: continue` or `break` is skipped in favour of the one before it; a loop that ran zero passes writes no last key, so a read of it renders empty rather than serving a stale value. Prefer last over guessing an N — for a while loop, and for a data-dependent each loop, N is unknowable.
- **iter_<N> inside the body is a trap.** N is a constant, so it does not render empty — it renders *pass N's* output during every pass, which looks right on pass N and is stale on every other. `cof` check warns.
- **prime.<loop>.<step>.value does not resolve, by design.** `prime.<loop>` is the loop's own node — `iter_<N>`, `last`, `collected`, `value`, `meta` — and never holds body step names. `cof` check warns.
- **The bare form `{{<step>.value}}` also works** and means the same node; it is accepted, not preferred, because a bare name can collide with a caller-supplied key.
- In CEL the same forms apply with the state. `prefix: state.prime.<step>.value`.

```
## △ validates with a warning – iter_0 inside the body pins every pass to the first
- type: loop
  name: courses
  each: {in: input.menu, as: course}
  body:
    - type: prompt
      name: prep
```

```

template: "List the prep for: {{course}}"
- type: prompt
  name: cook
  template: "Cook, given this prep: {{prime.courses.iter_0.prep.value}}"

```

The correct within-pass read is `{{prime.prep.value}}`. [Troubleshooting State Paths](#) has a symptom table — what `meta.prompt_sent` looks like for each wrong form — for the day one slips through.

7.6 Named or not

A **named** loop records everything above: `iter_<N>` per pass, `last`, `collected`, and a summary at the node itself:

```

prime.courses.value.iterations          # passes completed
prime.courses.value.termination.reason  # why it stopped (below)
prime.courses.value.effects_by_iteration # what each pass ran
prime.courses.meta.mode                 # "each", or the while condition's "model" /
↳ "cel"
prime.courses.meta.max_iterations / min_iterations # max_iterations only when set
prime.courses.meta.each_in_path / each_as        # each loops
prime.courses.iter_0.cook.value
prime.courses.last.cook.value
prime.courses.collected.value

```

In a saved shadow state — `--out`, `--json`, the `--live-state` mirror — `last` is not a second copy of the final pass. It is written once as a reference, `"last": {"$ref": "iter_2"}`, and every reader that loads the file links it back. [Shadow state](#) has the details. Inside the run, `prime.courses.last.cook.value` reads the same as always.

An **unnamed** loop is transparent: the body writes at stable paths in the enclosing scope, and each pass overwrites the last. No `iter_<N>`, no `last`, no `collected`. It is the right shape when only the final pass matters and nothing downstream needs the history. `collect` on an unnamed loop has nowhere to write and silently aggregates nothing:

```

## X runtime - no name, so no collected node: the array downstream expects is never
↳ written
- type: loop
  collect: cook
  each: {in: input.menu, as: course}
  body:
    - type: prompt
      name: cook
      template: "Write the cooking steps for: {{course}}"

```

Termination reasons — every completed named loop records one: `collection_exhausted` (every element done), `condition_false` (the while said stop), `max_iterations_reached` (the cap you set: a while that never said stop, or a truncated each, which also records `unvisited`), `collection_unresolved` (each.in was not an array), `condition_error` (the while condition could not be evaluated, under `break` or `continue`), `error` (the loop failed; `termination.detail` and `meta.error` say why). They are CEL-readable — `state.prime.season.value.termination.reason == 'max_iterations_reached'` is a fine thing to branch on after a taste loop that never converged.

7.7 Errors in a loop

on_error on a loop governs a failed *pass*: fail (default) propagates; break ends the loop at the failed pass, keeping what completed before it; continue drops the pass and goes on. In a tree loop every pass is already running, so break and continue both let the other passes finish and drop the failed ones. Under break and continue alike, last is the last pass that completed, never the failed one, and under break in a chain loop the failed pass is absent from collected.

Under continue, and under break in a tree loop, collected is not yet that clean. The runtime assembles it from the first k passes, where k counts the passes that completed. So the failed pass leaves a null in the array, and an each loop loses one pass from the end for each failure: courses [soup, salad, roast] with the salad failing collect as [soup-steps, null]. Until that is fixed, read the iter_<N> nodes when such a loop has had failures; each failed pass is an iter_<N> whose step has a null value and an error in meta.

7.8 Inside nested containers

{{course}} and {{_loop_index}} reach every effect in the body however deeply it is wrapped — an if branch, a grouping dynamic, an inner loop. An inner loop's as shadows the outer's if they share a name; give them different names. A CEL condition in the body reads the same two as state.course and state.iter.index, and a branch step reads the branch's earlier steps, as If describes:

```
- type: loop
  name: courses
  each: {in: input.menu, as: course}
  body:
    - type: if
      if: {mode: cel, expr: "state.input.diet == 'vegetarian'"}
      then:
        - type: prompt
          name: swaps
          template: "List vegetarian swaps for: {{course}}"
        - type: prompt
          name: cook
          template: "Write the cooking steps for course {{_loop_index}}: {{course}}, with
↪ these swaps: {{prime.swaps.value}}"
      else:
        - type: prompt
          name: cook
          template: "Write the cooking steps for course {{_loop_index}}: {{course}}"
```

An inner loop can iterate a field of the outer loop's element. each.in may start at an enclosing loop's as binding, the same way a CEL path or a use input {from: ...} can:

```
- type: loop
  name: courses
  each: {in: input.menu, as: course}
  body:
    - type: loop
      name: steps
      each: {in: course.steps, as: step}
      collect: do_step
      body:
        - type: prompt
          name: do_step
```

```
template: "For {{course.name}}: {{step}}"
```

With `input.menu` as `[{name: soup, steps: [chop, simmer]}, ...]`, each course's inner loop walks that course's own steps. A root that is neither a namespace nor a binding in scope fails `cof` check, and the error lists the bindings that are in scope.

7.9 Anti-patterns

Iterating prose. `each.in` on a text prompt. Make the producer an array prompt with a schema.

Reading collected or last from inside the loop. Both are written when the loop completes. Inside, read `{{prime.<step>.value}}`.

iter_0 inside the body. Stale data with a warning. Above.

Unbounded feedback. A model-mode while with no `max_iterations` and a question the model tends to answer *yes* to does not stop; with `max_iterations: 100` it is a hundred model calls. Set the cap to the number of passes you would accept, and make the condition ask for *stop* evidence, not *continue* enthusiasm.

A plural body. `summarize_articles` as a single prompt inside the loop is a sign the loop is not doing the iterating. One element, one singular step.

7.10 See also

- [Orchestration Reference](#) → [loop](#) and [Referencing a sibling within an iteration](#).
- [learn/loop](#) · [patterns/critique_refine_loop](#).

Chapter 8

Reflector

“Control mechanisms that lay their own plans.” — Gordon Pask, *An Approach to Cybernetics* (1961)

if chooses between plans that were written in advance. loop repeats one. The **reflector** writes the plan: it asks a model to *generate* the effects to run next, validates what came back against the same schema every hand-written document passes, executes it, and — if asked — does it again with the results in hand. It is the system planning itself, and it is the effect to reach for only when the steps cannot be known up front.

That last clause is the whole design guidance. A reflector costs a planning call per cycle, produces a topology you did not review, and is bounded only by the limits you set on it. When you *can* write the steps, write them: a chain is cheaper, faster, and auditable before it runs. The reflector is for the roast that burned an hour before the guests arrive — the recovery plan depends on what is in the kitchen now, and no one could have written it yesterday.

8.1 The shape

```
Reflector ::= { type: 'reflector', name: NAME, effects: Effect+,    - the planning dynamic,
  ↪ run each cycle
    plan_from_step?: NAME,                - default propose_steps
    max_iterations?: INT≥1,                - planning cycles; default 1
    max_effects?: INT≥1,                  - effects per plan; default 8
    stop_on_done?: BOOL,                  - default true
    generated_key?: STRING,                - default generated
    prime_template?: STRING, flow?: Flow, description?: STRING }
```

name and a non-empty effects list are required. Among the inner effects, one prompt — propose_steps by default, or whatever plan_from_step names — is the planner: its output is the plan.

8.2 A reflector in the kitchen

```
- type: reflector
  name: replan_service
  max_effects: 4
  max_iterations: 2
  effects:
    - type: prompt
```

```

name: propose_steps
template: |
  The roast burned. Six guests arrive in one hour for an anniversary dinner.
  Plan the final hour of service. Output must follow the OUTPUT CONTRACT exactly.

```

Each planning cycle runs in five phases:

1. **Render the prime directive.** A planning instruction — Circuitry ships a versioned default, `REFLECTOR_PRIME_V1` — is rendered with `max_effects` and two slots, a **goal** and a **context**. In the current runtime both slots render empty; [what the planner can see](#) below explains why.
2. **Run the inner dynamic** with the rendered prime prepended to the planning prompt's template. The planner sees the directive, then whatever the template says — which is why the example above writes the situation into the template itself.
3. **Extract the plan** from the planning step's output: a YAML document with a done flag and an effects list. Code fences are tolerated; markdown is not.
4. **Decide whether to stop.** An empty effects list stops. `done: true` stops when `stop_on_done` is set (the default) — *without executing that cycle's effects*. `done` means “there is nothing left to do”, so a planner that still has work to run must say `done: false`; the last useful plan is always a `done: false` one, and the cycle after it says `done: true` with nothing to run.
5. **Validate and execute** the plan through `use(inline)` — full schema validation, the same allowlists, the same cycle guards — as a state-isolated child under `prime.<name>.generated.iter_<cycle>`. Then, if cycles remain, plan again.

After `max_iterations` cycles, or an earlier stop, the reflector's own value is `true`. Any cycle that fails — the planner errored, the plan would not parse, the generated orchestration failed — fails the reflector (`value: false`, the reason recorded in `meta.iterations`) and with it the run: a reflector has no `on_error` of its own, so put one in its own document and call it through a `use` with `on_error: skip` if a failed plan must be survivable.

8.3 What the plan looks like

The directive fixes the output shape, and it is deliberately narrow:

```

done: false
effects:
- type: prompt
  name: warm_plates
  template: "Warm four plates. Keep the oven at the lowest setting."
- type: dynamic
  name: rescue_roast
  flow: chain
  effects:
  - type: prompt
    name: assess_roast
    template: "Describe the burned roast. Name the parts that are safe to serve."
  - type: prompt
    name: carve_plan
    template: "Write a carving plan for the safe parts."

```

Every generated effect is an ordinary effect — the plan is a Circuitry orchestration, and it passes the same gate as one you wrote. `max_effects` caps the number of top-level effects per cycle (8 by default; the alias `max_steps` is accepted).

The directive also fixes the plan's *language*: [ASD-STE100 Simplified Technical English](#) — one

instruction per sentence, active voice, imperative mood, a twenty-word ceiling on procedural sentences, one meaning per word, no vague verbs, articles never dropped — applied to the plan’s step descriptions and to every generated template. A narrowed output distribution is what makes generated plans predictable to validate and small planner models sufficient to write them. `prime_template`: replaces the directive wholesale, and with it the constraint; it is the author’s choice to keep it.

8.4 Isolation, and what the plan can see

A generated plan runs as a child with **isolated state**: like any use child, it cannot read the parent’s `prime` or `input` namespaces. A generated template that references `{{prime.courses.collected.value}}` renders empty. This is by design — the plan is a self-contained document, and everything its prompts need must be in their templates — and it is why the planning template should carry the specifics: the planner writes them into the plan.

The parent, on the other hand, sees everything the plan produced:

```
prime.replan_service.value           # true when planning finished
prime.replan_service.meta.iterations # one record per cycle: done, stop,
  ↳ parsed, error, plan_text
prime.replan_service.inner.propose_steps.value # the last planner reply
prime.replan_service.generated.iter_0.warm_plates.value # a generated effect's output
prime.replan_service.generated.iter_0.rescue_roast.carve_plan.value
```

Generated effects are addressable at exactly the paths their names dictate, one level under the cycle they ran in — so a downstream prompt can read a plan’s results by name once it knows the plan’s vocabulary, and a use effect elsewhere can wrap the reflector to map specific outputs.

8.4.1 What the planner can see

Today, three things: the directive, the literal text of its own template, and the reflector’s own node. Two things that look like channels into the rest of the run are not:

- **The directive’s goal and context slots.** They are meant to carry the value of a root-level effect named `goal` and the run’s effective settings. The directive looks both up in the store the reflector runs in, which holds `prime` rather than the state root, so it finds neither and both slots render empty. A `goal` effect before the reflector is harmless, and the planner never reads it.
- **Root references in the planning prompt.** The inner planning dynamic is rendered against the reflector’s own node rather than the run’s root, so `{{prime....}}` and `{{input....}}` written into `propose_steps` render empty too.

The node itself is readable, with paths relative to it. From the second cycle on, `{{{inner.propose_steps.value}}}` is the previous plan (triple-stache: it is YAML), and `{{generated.iter_0.warm_plates.value}}` is what one of its effects produced. That is how a planner works “with the results in hand”: name what the last cycle ran, and ask what comes next.

So write into the planning template, as plain text, everything the planner must know about the rest of the run. That keeps a reflector honest about what it is: a planner for a situation you can describe when you write the document, not one that reads the run. When the plan must depend on what the run produced, generate the plan with an ordinary prompt, which can read state, and run it with a use effect’s `inline:`, as [Composition](#) shows.

8.5 Bounding the feedback

Three limits, all worth setting explicitly:

- `max_iterations` — planning cycles. Each cycle is a planning call *plus* everything the plan runs.
- `max_effects` — the plan’s width.
- `stop_on_done` — whether the planner’s own `done`: `true` ends the loop. Turn it off only when you want exactly `max_iterations` cycles regardless.

And one switch outside the document: a [profile](#) with `effects.replan_service.enabled: false` turns agentic planning off for a single run, writing a skip node in its place. It is the effect most worth switching off — for a cost-controlled rerun, a deterministic test, a demo where the plan should not vary.

```
## profiles/no-planning.yml
effects:
  replan_service:
    enabled: false
```

8.6 Anti-patterns

A reflector where a chain would do. If you can name the steps, name them. The reflector’s cost is a plan you did not review.

A planning template with no specifics. “Plan the next steps” is all the planner gets, so it plans generic steps. Write the situation into `propose_steps`, as plain text.

A plan that reads parent state. Rendered empty; the plan must carry its own specifics.

Unbounded cycles. `max_iterations: 10` with `stop_on_done: false` is ten plans and everything they run. Set the number you would accept.

Replacing the prime casually. `prime_template` opts out of the output contract and the STE constraint at once. Keep the contract if you keep anything.

8.7 See also

- [Orchestration Reference](#) → [reflector](#).
- `learn/reflector` · `REFLECTOR_PRIME_V1` in `src/circuitry/core/primes.py`.
- [Decomposition](#) — the runtime’s own reflector: a planner that splits over-complex prompts.

Part III

The machine in the world

Chapter 9

Composition

“Observers are men, animals, or machines able to learn about their environment.”
— Gordon Pask, *An Approach to Cybernetics* (1961)

Part II closed the loop inside one document. Part III is the machine meeting the world, and the first thing it meets is *other documents*. An orchestration that does one thing well — critique a draft, extract fields, pick one of several candidates — is worth calling from other orchestrations, and calling it should be as safe as calling a function: it gets the inputs you hand it, nothing else; it hands back the outputs it declared, nothing else. That is the use effect, and the contract it calls through is the interface.

Orchestrations as organs. The curation library is the organ bank. And — the reason this chapter sits where it does — composition is the prerequisite for [decomposition](#), where the runtime uses the same mechanism to run plans it wrote itself.

9.1 use — the composition effect

```
Use ::= { type: 'use', name: NAME,  
  ref: LIBRARY_REF @ path: FILE_PATH @ inline: TEMPLATE,  
  validate?: BOOL,                - default true; gates inline YAML  
  inputs?: { NAME: value | TEMPLATE | {from: PATH} ... }, - become the child's  
    ↳ input.*  
  outputs?: { NAME: OutputDecl ... }, - omitted ⇒ full child namespace  
    ↳ exposed  
  on_error?: 'fail'|'skip'|'continue', description?: STRING }
```

```
LIBRARY_REF ::= '<category>/<name>' | '<source>:<category>/<name>'
```

A use runs another orchestration as an isolated sub-step. The child runs in its own store: only the mapped inputs pass in, only the mapped outputs come back. Exactly one of ref, path, or inline says which document.

```
- type: use  
  name: sauce  
  path: ./make_sauce.yml          # a sauce is a sub-recipe: its own document  
  inputs:  
    base: "{{prime.main_course.value}}"  
    style: pan  
  outputs:  
    recipe: {path: prime.compose.value, type: string}
```

inputs is a map of child input name to value; string values are Mustache-rendered in the parent before they cross. Inside the child they are `input.base` and `input.style`, indistinguishable from values a caller would have passed on the command line.

A rendered string always arrives as text, so a list interpolated with `{{...}}` crosses as its string form. To pass a value as it is — an array, an object, a number — write a reference instead of a template: `courses: {from: prime.menu.plan_courses.value}`. The path is rooted at `input.`, `prime.`, or `runtime.`, or at an enclosing loop's binding (`{from: course}` inside a loop over `courses`), and `cof` check rejects any other root. A reference that resolves to nothing passes null, and if the child's interface marks that input required, the use fails and names the path. The child cannot see the parent's `prime` — there is no path from inside `make_sauce.yml` to the parent's `main_course`, which is the point: a child that could read its caller's state could not be reasoned about on its own.

outputs is a map of parent-side name to a path *in the child*: `{path: prime.compose.value, type: string, description: ...}`. When present, `prime.sauce.value` is a flat dict of those names — `{{prime.sauce.value.recipe}}` downstream. (A bare string, `recipe: prime.compose.value`, is accepted shorthand for `{path: ...}`; write the object form — it is the one the library uses and the one with somewhere to put a type.)

9.1.1 Three ways to name the child

ref: — a library lookup, slash-delimited. `ref: utilities/critique` resolves through every configured [library source](#) in precedence order and takes the first match; `ref: hub:utilities/critique` resolves in exactly the source named `hub` and never falls through. This is the form for anything in the library. Every resolved `ref` is recorded in `runtime.library_refs` (and on the effect as `meta.library_ref`) with its source, path, and — for a GitHub source — the commit SHA it was served from, so a later run can reproduce the identical tree with no network at all.

path: — a filesystem path: absolute, working-directory-relative, or relative to the parent orchestration's own directory. For project-local helpers that are not library entries.

inline: — a Mustache template that renders to orchestration YAML at run time. This is how an LLM-generated plan runs: a prompt writes the document, and a use executes it, after validating it against the full schema (`validate: true`, the default — keep it):

```
- type: prompt
  name: plan_prep
  template: "Write a Circuitry orchestration YAML that preps the kitchen for:
    ↪ {{input.dish}}. Output YAML only."

- type: use
  name: run_prep
  inline: "{{{prime.plan_prep.value}}}"
  validate: true
```

Triple-stache matters here: the plan is YAML, and `{{...}}` would HTML-escape its quotes. An inline child is the mechanism under the [reflector](#) and under [decomposition](#); they add the planning directive and the bookkeeping, and delegate the execution to exactly this.

(A fourth field, `orchestration:`, is a deprecated hybrid of `ref` and `path`. It still parses and warns. Write `ref` or `path`.)

9.1.2 Output modes

Mode	When	What lands
Declared outputs	the use has <code>outputs:</code> , or the child declares <code>interface.outputs</code>	<code>prime.<use>.value</code> is a flat dict of the declared names
Full namespace	neither	the child's entire prime subtree at <code>prime.<use>.<child_effect>.value</code>

An explicit `outputs:` on the use wins over the child's `interface.outputs`. Full-namespace mode is convenient for a quick project-local child and wrong for a library entry: a caller that reaches into `prime.critique_step.critique.value` is coupled to the child's internal effect names, which is precisely what the interface exists to hide.

9.2 interface — the contract

```
Interface ::= { inputs?: { NAME: InputDecl ... },
               outputs?: { NAME: OutputDecl ... } }
InputDecl  ::= { type?: 'string'|'number'|'boolean'|'array'|'object', required?: BOOL,
               ↪ description?: STRING }
OutputDecl ::= { path: STATE_PATH, type?: STRING, description?: STRING }
```

An orchestration declares its contract with a top-level interface: `block` — the child's mise en place, everything it needs laid out and named before any heat is applied:

```
## make_sauce.yml
interface:
  inputs:
    base:
      type: string
      required: true
      description: The dish the sauce accompanies.
    style:
      type: string
      required: false
  outputs:
    recipe:
      type: string
      path: prime.compose.value

effects:
- type: prompt
  name: compose
  template: "Compose a {{input.style}} sauce for {{input.base}}."
```

Two things happen when a use calls a document with an interface. **Required inputs are validated** — a caller that omits `base` fails with *missing required input 'base' declared in orchestration interface* before the child runs a single effect. And **outputs are auto-generated**: the caller's mapping is the child's `interface.outputs`, so callers do not repeat dot-paths, and the child can rename its internal effects without breaking anyone. The use in the first example could drop its `outputs:` block entirely.

The interface is also what `cof info` prints, what `cof list` shows as inputs, and what the [wizard](#) reads when it composes library entries into a draft. Declare one on anything reusable-shaped.

A declared input has one spelling inside the document — `{{input.base}}` in templates, `state.input.base` in CEL — and the compiler holds you to it:

```
## X a declared input read bare is a compile error
interface:
  inputs:
    base: {type: string, required: true}
  effects:
    - type: prompt
      name: compose
      template: "Compose a sauce for {{base}}."
```

9.3 Isolation and observation

Isolated *state*, shared *observation*. The child's effects are reported to the parent run's observers — `--live-state`, the TUI, every runtime plugin's `on_effect_start` / `on_effect_complete` — at paths namespaced under the use node: a child effect `compose` inside `use:` `sauce` announces as `prime.sauce.compose`, and a use inside a use composes the same way. Watchers see the whole tree unfolding; what actually lands in parent state is still exactly what the output mode says.

Cycles are rejected. The compiler walks the static use graph across sources — A in a folder, B in the hub cache, A again — and refuses at validation. The runtime keeps a per-execution stack of resolved paths and hashes rendered inline YAML, so a cycle that closes only at run time is caught too. Two utilities that legitimately need each other are a sign of a third utility they both need.

Unfetched sources fail early. A ref into a GitHub source whose cache was never populated fails at validation, naming the command that fixes it (`cof library refresh hub`) — never mid-run. A genuinely unknown ref (a typo) surfaces as the use effect's own error when it runs.

Errors inside the child are the child's, under its own effects' `on_error`; a child that fails as a whole fails the use, under the use's `on_error`. Nothing partial ever lands in the use node's value — a failed child's scratch state is discarded.

The use node keeps a record of the call either way. `meta.inputs` is what the child actually received, `meta.orchestration_sha256` is the SHA-256 of the child's YAML, and `meta.child_errors` lists, as `{path, error}`, every effect inside the child that failed and was skipped under its own `on_error` — so a composed call that went green still says what it lost (null when nothing did). To keep the child's own effects in the final state as well, set `runtime.state.record_children: true` in config. Each use node then holds its child's effects beside the mapped value, at every depth, a failed child's partial record included. That is for the audit trail: downstream effects still read the declared outputs, never the recorded child effects.

9.4 The library

`ref:` resolves through `runtime.library.sources`, an ordered list where **order is precedence**. With no configuration the list is `[{type: curation}]`: the bundled library, organised by category.

Category	What lives there
learn/	single-primitive demonstrations, one concept per file — the first read

Category	What lives there
utilities/	composable, single-output orchestrations with interfaces: summarize, critique, refine, judge, classify, decompose, extract, route
patterns/	multi-primitive templates: critique_refine_loop, parallel_then_judge, classify_then_route, all_primitives
recipes/	full workflows: article_summarizer, research_brief, code_review, meeting_notes, comic_strip
agents/	orchestrations that build or improve orchestrations: wizard, meta_orchestrator, improver, improver_judge, decompose

`cof list` browses it; `cof info recipes/article_summarizer` shows an entry's interface; `cof eject recipes/article_summarizer` copies it into the working directory for editing; `cof run utilities/critique -e content=... -e criteria=...` runs one directly.

Two more source types extend the list. A **folder** source scans a directory of your own *.yaml files — entry names are paths relative to the folder root, categories are its subdirectories, and an optional `manifest.json` adds descriptions and inputs. A **github** source serves a subtree of a repository from a SHA-pinned local cache; `cof library refresh <name>` is the only command that ever touches the network, so a run can never stall on a fetch or silently pick up a different version than the one you listed. This is the consumption half of a publish-by-PR shared library.

```
{
  "runtime": {
    "library": {
      "sources": [
        {"type": "curation"},
        {"type": "folder", "name": "local", "path": "./orchestrations"},
        {"type": "github", "name": "hub", "repo": "owner/name", "ref": "main", "path":
          ↪ "library/", "token_env": "GITHUB_TOKEN"}
      ]
    }
  }
}
```

[Library Sources](#) has the full field table; [Shared Library](#) and its companions cover the contribution workflow and the older `cof fetch` / `cof run-library` retrieval commands.

9.5 Composing the dinner

The utilities are written to chain. A draft menu, critiqued against named criteria, refined on the critique:

```
- type: prompt
  name: draft_menu
  template: "Draft a three-course menu for {{input.occasion}}, one line per course."

- type: use
  name: critique_step
  ref: utilities/critique
  inputs:
```

```

    content: "{{{prime.draft_menu.value}}}"
    criteria: "balance, seasonality, brevity"

- type: use
  name: refine_step
  ref: utilities/refine
  inputs:
    content: "{{{prime.draft_menu.value}}}"
    feedback: "{{prime.critique_step.value.critique.issues}}"

- type: prompt
  name: menu_card
  template: "Write the menu card from: {{{prime.refine_step.value.refined}}}"

```

critique declares one output, critique (an object with score, issues, strengths), so `prime.critique_step.value.critique.issues` is the list of things to fix; refine declares refined. Neither caller repeats a path into the child, and either utility can be reworked internally without touching this document. `patterns/critique_refine_loop` wraps the same two calls in a while loop with the model judging when to stop.

9.6 Anti-patterns

Passing state by reaching. A ref pointing at a state path, or a child template that expects `{{prime.<parent_effect>.value}}`. The only door is `inputs:`.

Full-namespace coupling to a library entry. Declare outputs, or rely on the child's interface.

A use with prompt fields. `template`, `schema`, `prompt_type` do not belong on a use. If the step is a model call, it is a prompt.

Naming the effect use. Name it for what the composed step does — `critique_step`, `sauce`.

Re-deriving a utility inline. If a library entry's interface fits, call it; the entry has been tuned and its output shape is stable.

9.7 See also

- [Orchestration Reference](#) → [use](#) and [Outputs](#).
- [Library Sources](#) · [Shared Library](#).
- [patterns/parallel_then_judge](#) — a tree of candidates and a judge utility choosing one.

Chapter 10

Complexity — scoring and routing

The system measuring itself. Every prompt in a run has a difficulty — a two-line classification is not a forty-reference audit — and a runtime that can *measure* that difficulty can act on it: send the easy prompt to a small, fast, cheap model and the hard one to a large one, and say afterwards which it chose and why. Circuitry does this with three independent switches, all off by default: **scoring** measures, **routing** chooses a model from the measurement, and **decomposition** splits what is too big for any one model to do well.

This chapter is the first two. Nothing in it requires a model call: scoring is deterministic and mechanical, and routing is a table lookup.

10.1 Scoring

With `runtime.complexity.scoring.enabled: true`, every prompt effect is measured on seven signals before it dispatches. Each signal is normalised to `[0, 1]`, multiplied by its weight, and summed; the total is rescaled onto a **0-100** score so that band tables and thresholds are portable between orchestrations whatever their weights.

Signal	Default weight	Measures
<code>prompt_size</code>	1.0	Token estimate of the prompt — the template before a run, the rendered text once state is interpolated.
<code>state_references</code>	1.5	Distinct <code>{{...}}</code> references the template reads.
<code>prompt_type</code>	0.75	How structured the expected output is: <code>boolean/number</code> lowest, <code>text</code> in the middle, <code>json/object/array</code> highest.
<code>output_schema</code>	1.25	Depth and field count of a declared schema.
<code>output_size</code>	1.0	Expected output size — a schema's <code>maxItems</code> , other declared limits.

Signal	Default weight	Measures
structural_position	0.5	Nesting: inside a loop body, inside a conditional branch, generated by a reflector.
keywords	1.0	Whole-word, case-insensitive matches against a keyword → weight table.

Mechanical signals only. The scorer does not ask a model how hard the prompt is; it counts. That is what makes it free, deterministic, and — because the same seven names are what you configure, what the runtime records, and what the TUI prints — explainable end to end.

```
{
  "runtime": {
    "complexity": {
      "scoring": {
        "enabled": true,
        "weights": {"state_references": 2.0},
        "keywords": {"audit": 8, "reconcile": 6}
      }
    }
  }
}
```

weights overrides only the signals you name; an unknown name is a config error at resolution time, never a silent no-op. keywords is the phrase table itself — and its documented default of {} is easy to misread as *off*: left unset, a built-in table of reasoning-shaped verbs (analyze, classify, compare, summarize ...) is active, and a table you supply replaces it.

10.1.1 What scoring writes

Every scored prompt's node gains meta.complexity, written *before* dispatch — so it is on the node when the prompt fails, which is when you most want to know how hard it was, and on a --dry-run node too:

```
"meta": {
  "model": "llama3.1:8b",
  "model_reason": "default",
  "complexity": {
    "score": 34.7,
    "max_score": 100.0,
    "mode": "rendered",
    "signals": {
      "prompt_size": {"raw": 412.0, "normalized": 0.34, "weight": 1.0, "contribution":
        ↳ 4.86, "note": "~412 tokens of rendered prompt"},
      "state_references": {"raw": 3.0, "normalized": 0.3, "weight": 1.5, "contribution":
        ↳ 6.4, "note": "3 distinct references"}
    },
    "warnings": []
  }
}
```

With scoring off there is no complexity key at all — not null, not {} — so a run with the switch off produces the same state tree as a build without the feature. And because signals is keyed by name, the score is **addressable from CEL**. An orchestration can branch on how hard its own step was:

```

- type: if
  name: review_depth
  if:
    mode: cel
    expr: "state.prime.plan_courses.meta.complexity.score > 60.0"
  then:
    - type: prompt
      name: review
      template: "Review this menu plan in detail, course by course:
↪ {{prime.plan_courses.value}}"
    else:
      - type: prompt
        name: review
        template: "Sanity-check this menu plan in two lines: {{prime.plan_courses.value}}"

```

10.1.2 Previewing: cof score

```

$ cof score dinner_party.yml --config bands.json
Circuitry · Score (static preview)

```

Effect	Type	Score	Band	Dominant signals / reason
draft_menu	prompt	7.3	light	state_references 4.3
critique_step	use	—	—	not scoreable — compiled at runtime
menu_card	prompt	7.2	light	state_references 4.3

2 scored, 1 not scoreable, 3 effect(s) total.

cof score compiles the document and scores each prompt's *template* with no state available — a static preview, no model calls. Five signals are identical either way; prompt_size and keywords move once real state is interpolated, usually upward. Four kinds of effect cannot be scored at all and are reported as rows with a reason rather than dropped: a use (its child is not compiled until it runs), a tool (no prompt), a reflector-generated effect (does not exist yet), and an effect a profile disabled. --json gives the same table as data.

10.2 Routing

With routing.enabled: true, each prompt's score picks a row from an ordered **band table**, and that row's model is what dispatches:

```

{
  "runtime": {
    "complexity": {
      "scoring": {"enabled": true},
      "routing": {
        "enabled": true,
        "respect_explicit": true,
        "bands": [
          {"name": "light", "max": 20, "model": "phi3:mini"},
          {"name": "mid", "max": 40, "model": "qwen2.5:7b-instruct"},
          {"name": "heavy", "max": 60, "model": "gpt-oss:20b"}
        ]
      }
    }
  }
}

```

Bands are ordered; max is the band's **inclusive** upper bound (a score of exactly 40 lands in mid); the final band omits max and is the required **catch-all**, so every score resolves to a model. The table is validated at config resolution — an empty table, a missing or misplaced catch-all, non-ascending bounds, a max outside 0-100, an unknown key — all fail before the first effect dispatches, and they fail even when routing is *disabled*, so a broken table surfaces before you flip the switch.

Model names pass through untouched. For a local adapter they are model names; for the CyberDiner broker they are capability tiers (cheap, good, alpha); the router resolves a band to a string and does not interpret it. A band table is also, by itself, a sufficient model configuration: with routing on and no other model set anywhere, the catch-all becomes the run default, and effects that cannot be scored run on it.

10.2.1 Where the router sits

The router only ever substitutes the **run default**. Anything a human named on purpose beats it:

```
a profile's per-effect model > the effect's own model: > --model > a profile's
↳ run-level model:
  > a profile's per-effect routing pin > ROUTER > the document's model: > config
  ↳ default_model
```

respect_explicit: true (the default) is that rule. Set it to false and the router decides for every scored prompt, including ones that name their own model and runs launched with -model — the knob for a cost sweep or a bulk re-run, off by default because overruling a deliberate choice is the surprising behaviour.

A [profile](#) can adjust one effect without touching the table: routing: false opts the effect out (it dispatches on the model it would use with routing off entirely), and routing: heavy pins it to a named band, bypassing the score. A pin does not need scoring on — it names a model directly — and an explicit model: still beats a pin.

10.2.2 Provenance

A routed effect records who decided:

```
prime.<name>.meta.model          # the model that dispatched
prime.<name>.meta.model_reason    # "router" | "explicit" | "default"
prime.<name>.meta.complexity.band # {name, model} — the row the score fell in
runtime.effective_settings.sources.model # "router" when the router won the run
```

band is recorded even when the router *defers* — on an effect that pins its own model, or one a profile opted out — answering “what would routing have picked?”, which is the question you ask right before removing a pin. model_reason is the field that says whether the band was applied; band alone never implies it was.

10.2.3 Watching it happen

```
$ cof run dinner_party.yml --explain-routing -e occasion=anniversary
▶ prime.classify score 7.2/100 · band light · model phi3:mini · why router
▶ prime.plan_courses score 20.5/100 · band mid · model qwen2.5:7b-instruct · why router
▶ prime.audit score 41.7/100 · band heavy · model llama3.1:70b · why explicit
```

One line per prompt effect, printed the moment before it dispatches, read straight off the node's pre-dispatch meta — score, band, model, and why. The third line is the router deferring; audit names its own model and still reports the band it scored into. With scoring off the flag prints nothing; --quiet and --json suppress it.

`docs/examples/routing/` is a runnable demonstration — one orchestration, two band tables, a pin-and-opt-out profile — that runs under `--dry-run`, because routing happens in the pre-dispatch meta block and no model is ever called.

10.3 Per-run switches and precedence

```

cof run dinner_party.yml --scoring --routing           # force both on for this run
cof run dinner_party.yml --no-routing                 # off, whatever config says —
  ↪ beats a profile pin too
cof run dinner_party.yml --scoring --decompose --dry-run # try decomposition without
  ↪ editing config

```

Each flag flips one enabled field; the rest of the sub-block (weights, bands, thresholds) is whatever config or the document already set. The block itself rides the normal runtime.* precedence — a document’s `runtime.complexity` **replaces** the config’s wholesale rather than merging, so a document that overrides it must restate everything it still wants:

```

--scoring/--routing/--decompose > orchestration runtime.complexity > config
  ↪ runtime.complexity > defaults

```

`runtime.effective_settings.sources.complexity` (and `.scoring`, `.routing`, `.decomposition`) records which layer won.

Routing and decomposition each require scoring — enabling either without it is a config error naming the prerequisite — but not each other: score only, score + route, score + decompose, and all three are each valid.

10.4 Anti-patterns

A table without a catch-all. Rejected at resolution. Every score needs a model.

respect_explicit: false in a shared config. It overrides every deliberate `model:` in every document that runs under it. Reserve it for the run where that is the point.

Reading band as “was routed”. `model_reason` is the answer.

Trusting the static preview as the runtime score. `cof score` measures templates; the run measures rendered prompts, which pull in whatever state they reference. Use `--explain-routing` on a `--dry-run` to see the rendered numbers before spending anything.

A document-level `runtime.complexity` that sets one key. It replaces the whole block; restate the rest.

10.5 See also

- [Complexity Configuration](#) — the full schema, every error message, `meta.decomposition`.
- [Routing](#) — signals, band tables, the full precedence ladder, a worked example, and what the scorer honestly cannot see.
- [cof score](#).

Chapter 11

Prompt decomposition

Everything above, composed. “Cook Thanksgiving dinner for twelve” is too much for one prompt — a single model call asked to plan, cook, and plate a dozen dishes does all of them badly — and when that prompt scores over the decomposition threshold, the runtime *knows* it. What it does next uses every part of the language so far: the prompt goes to a **planner that is itself an orchestration** (chapter 8’s reflector, purpose-built); the planner emits a fan-out-and-merge document (chapter 2’s tree and chain); the document is validated and run as an isolated child (chapter 9’s use); the merged result is written back at the original prompt’s own state path (chapter 3’s determinism), so the effect after it reads `{{prime.<name>.value}}` and never knows dinner was ever in pieces.

This is the third of the three complexity switches, and it needs the first: decomposition consumes scores, so `scoring.enabled` must be on.

11.1 Turning it on

```
{
  "runtime": {
    "complexity": {
      "scoring": {"enabled": true},
      "decomposition": {
        "enabled": true,
        "threshold": 45,
        "max_chunks": 5,
        "max_depth": 1,
        "on_failure": "route_up"
      }
    }
  }
}
```

Field	Default	Meaning
threshold	80	A prompt whose score strictly exceeds this is decomposed. A score equal to it does not trigger. 0-100.
max_chunks	8	The widest fan-out a plan may propose.

Field	Default	Meaning
max_depth	2	Recursion ceiling: chunks are scored too, and an over-complex chunk decomposes again, down to this depth.
on_failure	route_up	route_up runs the original prompt on a stronger model instead; fail propagates the error.

cof run ... --decompose forces the switch on for one run (with --scoring, or scoring already on); --no-decompose forces it off.

11.2 What a decomposition does

When a prompt’s recorded score exceeds threshold, the runtime replaces the single model call with three bounded steps.

1. Plan. The bundled planner — agents/decompose, a curation entry you can read and run yourself — runs in an isolated store. It is handed the effect’s raw template, a listing of the inputs the template reads, the effect’s declared output shape (prompt_type and schema), and max_chunks. Its directive (DECOMPOSE_PRIME_V1) is specific: *split on the seam that is already in the prompt* — one chunk per numbered instruction, per input document, per output field, per entity — and *do not invent a pipeline the work does not have*. The planner’s own run has decomposition switched off; its planning prompt would otherwise trigger the feature it implements.

2. Validate. The emitted YAML must parse, pass the full orchestration schema, keep the fan-out within [2, max_chunks] — a “decomposition” into one chunk is the original prompt with scaffolding and is refused — and honour the merge contract: a top-level effect named merge, producing the same prompt_type and schema as the source, with interface.outputs.result pointing at prime.merge.value, and interface.inputs declaring every input the source template read, under the same names. An invalid plan never runs.

3. Execute and write back. The plan runs as a state-isolated child seeded with a copy of the effect’s render context — the same inline-identity cycle guard as a use child, the same namespaced observability (chunk effects announce under the decomposing effect’s node, and live-state snapshots mirror them there). The value at prime.merge.value is written at the **original effect’s own path**. Downstream, nothing changes.

Thanksgiving, decomposed, looks like a plan you might have written yourself:

```
interface:
  inputs:
    guests: {type: number, required: true}
    dietary_notes: {type: string, required: false}
  outputs:
    result: {path: prime.merge.value, type: string}

effects:
- type: dynamic
  name: courses
  flow: tree
  effects:
```

```

- type: prompt
  name: turkey
  template: "Write the plan to roast a turkey for {{input.guests}} guests. Note
  ↳ timing."
- type: prompt
  name: sides
  template: "Write the plan for three side dishes for {{input.guests}} guests.
  ↳ Respect: {{input.dietary_notes}}"
- type: prompt
  name: dessert
  template: "Write the plan for two desserts for {{input.guests}} guests."
- type: prompt
  name: merge
  template: |
    Combine these plans into one timeline for the day, oven by oven.
    Turkey: {{prime.courses.turkey.value}}
    Sides: {{prime.courses.sides.value}}
    Desserts: {{prime.courses.dessert.value}}

```

Courses that cook in parallel, and a merge that plates them.

11.3 Bounded at every edge

Depth. Chunks are scored inside the child, so an over-complex chunk decomposes again — and `max_depth` bounds that recursion. At the ceiling the effect **routes up** rather than decomposing: it runs once on the routing table's catch-all (most capable) band model — or, with routing off, simply runs as-is — and the depth it stopped at is recorded. The ceiling is checked before the planner runs, so a depth-limited effect costs no extra calls.

Failure. `on_failure` governs the three real failure paths — the planner failing, an invalid plan, the child execution failing. `route_up`, the default, falls back exactly like the depth ceiling: the original prompt runs on the catch-all band model, the failure is recorded under `meta.decomposition.reason`, and the run lives. **A decomposition attempt can never turn a working run into a failed one.** `fail` propagates like any other effect error, under the effect's own `on_error`. Either way the child's scratch state is discarded whole; nothing partial ever lands at the original path. `respect_explicit` from the routing block covers the fallback too: an effect that pins its own model: keeps it when route-up would otherwise substitute one.

Record. `meta.decomposition` on the effect node carries the full decision — {`decomposed`, `outcome`, `reason`, `score`, `threshold`, `depth`, `max_depth`, `plan`, `chunk_count`, `yaml`, `result_path`, `fallback_model`, `error`} — whether it decomposed, routed up, ran as-is, or failed. Like `meta.complexity`, the key is absent entirely when the feature never triggered.

11.4 Reading the plans: --decompose-out

`meta.decomposition.yaml` carries the generated orchestration, but a run that finishes and is never looked at makes that plan effectively unreadable. `--decompose-out <dir>` writes every plan out, one file per decomposed effect:

```
cof run thanksgiving.yml --scoring --decompose --decompose-out ./plans -e guests=12
```

```
plans/
  3f2c9e7a-...__prime.plan_dinner.yml
```

Each file is `<run_id>__<effect_path>.yaml`, the planner’s YAML verbatim, headed by a comment block naming the run, the effect, the outcome, and the merge contract — plain comments, so `cof run ./plans/<file>.yaml` runs it as an ordinary document. Failed plans are written too, with `# status: failed` and the error: a plan that failed validation or whose execution failed is the one worth reading, and a planner payload that failed its own envelope is written as `.rejected.yaml`. This is the loop that grows the library — a good plan is a candidate recipe/ entry, promoted by hand.

11.5 Why the seams matter

Decomposition is where the design pays off all at once. The planner can emit a fan-out because the *dynamic* makes parallel composition a first-class thing. The runtime can run the plan safely because *use* isolates it and *validation* gates it. The merged result can be substituted invisibly because every effect’s path is *deterministic* — `prime.plan_dinner.value` means the same thing whether one model call or a child orchestration produced it. And the decision is auditable because everything that happened was written to meta. None of that was built for decomposition; decomposition is what you get when the pieces fit.

11.6 Anti-patterns

A threshold at the default with a small local model. 80 is conservative. If your model struggles at 45, set 45 — `cof score` shows you where your prompts sit.

Decomposition without routing. Legal, and the route-up fallback then “runs as-is” — meaning a failed decomposition retries the original prompt on the same model that was too small for it. Pair the two switches.

Expecting chunks to read the parent’s state. They read the *source effect’s inputs*, which the plan declares; the chunks are a self-contained document.

Never reading the plans. `--decompose-out` costs nothing. The plans are how you learn what the planner thinks your prompts are made of — and where the next library entry comes from.

11.7 See also

- [Complexity Configuration → decomposition](#) — every field, error, and the exact `meta.decomposition` record.
- [agents/decompose](#) — the planner, as an orchestration you can run.

Chapter 12

Surfaces — CLI, TUI, SDK, MCP, and observability

Everything the world reaches Circuitry through. One runtime sits under all of them — the CLI, the terminal UI, the Python SDK, the MCP server, and the REST trigger all call the same run path, so an orchestration behaves identically whichever door it came in by (except for the host settings in its runtime: block: a file you name by path applies them, a document a library, tool or network caller hands in does not — see [Configuration](#)) — and one record comes out of all of them: the state graph, which is the complete decision record of the run.

12.1 The CLI: cof

Command	What it does
<code>cof setup</code>	Interactive first-run: detect local backends, pick a model, write the global config.
<code>cof init</code>	Write a project config and a <code>hello.yml</code> in the current directory, and trust the config.
<code>cof trust [path]</code>	Show what a project config sets, flag the host-sensitive settings, and — after you confirm (<code>--yes</code> to skip) — trust it so runs apply it. <code>--list</code> shows every trusted file and whether it still matches. See Configuration .
<code>cof untrust [path]</code>	Stop applying a project config you trusted.
<code>cof doctor</code>	Run every extension's preflight check() and report, including the project config and its trust state; <code>--generate</code> also makes a live model call. Non-zero exit when anything fails.
<code>cof run <name-or-path></code>	Execute an orchestration. Library entries by slash name (<code>learn/hello</code>), files by path.
<code>cof check <path></code>	Validate against the schema and the static rules, then preflight the extensions it references. <code>validate</code> is the same command. <code>--skip-preflight</code> for structure only.
<code>cof score <name-or-path></code>	Static per-effect complexity preview; no model calls.

Command	What it does
<code>cof list</code>	Browse the library. <code>--extensions</code> lists compiled-in adapters, tool plugins, and runtime plugins; <code>--models <adapter></code> asks an adapter what it can serve.
<code>cof info <name></code>	An entry's description, interface, and resolved complexity settings with provenance.
<code>cof eject <name></code>	Copy a library entry into the working directory for editing.
<code>cof inspect <path></code>	Static metadata: effect counts and types, declared runtime hints.
<code>cof gen <name> "<goal>"</code>	Generate an orchestration from a sentence, single-shot, via <code>agents/meta_orchestrator</code> .
<code>cof wizard --goal "..."</code>	Build one by conversation — clarifying questions, then a validated draft — via <code>agents/wizard</code> .
<code>cof library refresh <source></code>	Fetch a remote library source into the local cache. The only library command that touches the network.
<code>cof fetch / cof run-library</code>	Retrieve and run a shared-library asset by id and version.
<code>cof mcp</code>	Run the MCP server on stdio (also <code>circuitry-mcp</code>).
<code>cof tui</code>	Launch the terminal UI (needs the <code>tui</code> extra).
<code>cof version</code>	

12.1.1 cof run

```

cof run learn/hello -e name=World           # inline inputs, repeatable
cof run dinner.yml --state inputs.json       # inputs from a file
cof run dinner.yml -e occasion=anniversary --tail # print only the final effect's
↳ value
cof run dinner.yml --json | jq '.prime.plate.value' # machine-readable state (automatic
↳ when piped)
cof run dinner.yml --out state.json --pretty      # write the finished state
cof run dinner.yml --live-state dinner.live.json # mirror the state while it runs (≤
↳ every 0.5 s, and at the end)
cof run dinner.yml --dry-run                   # no model calls; rendered prompts
↳ and shapes
cof run dinner.yml --verbose                   # a start and a result line per
↳ effect, pass tags, timing
cof run --last                                # re-run the previous invocation
cof run dinner.yml --profile fast              # apply profiles/fast.yml
cof run dinner.yml --profile-from-state runs/fast.json # reconstruct the profile a past
↳ run recorded
cof run dinner.yml --adapter ollama --model llama3.2 # the run default, highest-priority
↳ layer
cof run dinner.yml --scoring --routing --explain-routing
cof run dinner.yml --scoring --decompose --decompose-out ./plans
cof run dinner.yml --skip-preflight           # run even if a check() reported
↳ not-ready

```

When `stdout` is not a terminal, `cof run` switches to `--json` with quiet output on its own, so it composes with `jq` and with scripts without flags. `--tail` overrides that when you want the raw final value. `--quiet` suppresses prose; `--config <path>` (or `CIRCUITRY_CONFIG`) points at a specific config file.

`--verbose` prints one start line and one result line per effect. A step inside a loop carries its pass as `[N]`, and a child effect of a use is labelled with the use that called it. The adapter on a prompt's line is the one the call actually went to, with a failed primary shown before the fallback that answered. The same run also warns when a while loop stops at `max_iterations` rather than on its condition.

12.1.2 `cof gen` and `cof wizard`

Two ways to write an orchestration without writing YAML, and they are different artifacts. `cof gen` is single-shot: one goal in, one document out, driven by the `meta_orchestrator` agent with the full authoring ruleset injected. `cof wizard` is a conversation: the wizard agent handles one turn at a time — *interpret* → *ask-or-draft* → *validate* → *revise* → *gate* — and hands back either a question or a validated draft, with `done true` only when it is finished *and* the draft passed validation, so it cannot finish on invalid YAML. `--reply answers.txt` drives it headlessly. [The Wizard](#) documents the turn contract.

12.2 The TUI

`cof tui` is the same runtime with a keyboard: eight views, reachable by number (3, Inspect, is still a placeholder screen), every widget focusable without a mouse.

Key	View	
1	Library	browse entries, filter, read an interface, launch
2	Run	the launch form — inputs, profile, adapter and model, the three complexity switches as tri-state dropdowns — then live progress with a complexity column
4	Runs	the state inspector: a tree of every node in a finished run, values and meta side by side
5 / 6	Doctor / Settings	preflight results; the resolved config and where each value came from
7	Validate	<code>cof check</code> with the warnings channel visible
8	Chat	the wizard, conversationally
9	Profiles	edit a named profile: the effect tree with a picker and a toggle per row

? shows the help overlay; q goes back, then quits; Ctrl-C quits from anywhere. [Terminal UI](#) is the full tour.

12.3 The SDK

```
from circuitry import run_orchestration

result = run_orchestration(
    orchestration_path="dinner.yml",
    state={"occasion": "anniversary", "diet": "vegetarian"},
    live_state_path="dinner.live.json",
)
print(result.ok) # True / False
print(result.state["prime"]["plate"]["value"])
print(result.state["runtime"]["effective_settings"]["sources"])
```

`run_orchestration` reuses the CLI runtime path deliberately, so behaviour is identical across interfaces. `state` is wrapped under `input` for you. `dry_run=True` and `validate_only=True` do what the flags do; `out_path` writes the state; `raise_on_error=True` (the default) raises `CircuitryExecutionError` — which carries the failed `RunResult` as `.result`, so the partial state is never lost — and `False` returns the result with `ok` false instead.

`run_orchestration` trusts the file you pass the way `cof run ./file.yml` does: its whole runtime: block and plugins: list apply, with a notice in `result.warnings` when they include host settings. Pass `trust_document=False` for a path you did not choose — a fetched or generated file, or one a caller of your code named — and the document is limited to `runtime.complexity` and `runtime.state`. `run_shared_orchestration` is always limited.

The `adapter=` parameter is the seam: pass an already-constructed object implementing the Adapter protocol (`generate(model=, prompt=, timeout_seconds=)`) and the run uses it instead of the one config resolves, with preflight skipped. It is how a host supplies its own model transport, and how a test scripts one.

The rest of the public surface: `validate_orchestration(orchestration_path=)` returns `{ok, errors, warnings}`; `inspect_orchestration` returns static metadata; `inspect_divergence_paths(state=)` walks a finished state and returns every errored node in path order; `run_shared_orchestration` runs a shared-library asset by id and version; `circuitry.adapters.build_adapter` constructs an adapter from config. [API Reference](#) and [Stability](#) say what is public and how it is versioned.

Embedded callers also get the two per-effect events without writing a plugin: `RunRequest.effect_start_observer` and `RunRequest.effect_observer` are (`effect_path`, `effect_node`) callables, composed with any configured plugins.

12.4 The MCP server: a Claude session as the model

`circuitry-mcp` (also `cof mcp`) is an MCP server that lets a Claude Code or Claude Desktop chat *drive* an orchestration — with the host session as the LLM. Each prompt effect pauses the run and hands the rendered prompt to the assistant; the assistant's reply becomes the effect's value; tool effects still execute server-side.

```
{
  "mcpServers": {
    "circuitry": {"command": "circuitry-mcp"}
  }
}
```

The tool loop: `list_orchestrations()` → `run_orchestration(orchestration, initial_state)` returns `{run_id, status, pending_prompts, state}` → for each entry in `pending_prompts`,

`submit_response(run_id, prompt_id, response)` → repeat until status is completed, failed, or cancelled. `pending_prompts` is always a list: length one for a sequential run, length N for a flow: tree or a parallel loop — parallel branches simply arrive together, and submission order does not matter. `get_run_state(run_id)` snapshots mid-run; `cancel_run(run_id)` wakes every blocked branch. `validate_orchestration(path)` is there too. The model picks the path, so an MCP run or validation is limited like a library document: only `runtime.complexity` and `runtime.state` apply, and the response’s warnings name what was ignored.

Underneath is the `host_claude` adapter, which is why the plain `cof run` is unchanged and the MCP server is a strict addition. The bundled `/cof slash` command (`.claude/commands/cof.md`) teaches the loop to a Claude Code session; a `project.mcp.json` registers the server.

12.5 REST and scheduling

`circuitry.service` carries a minimal REST trigger — `POST /v1/triggers/run` with a state payload, bearer-token gated — and a scheduler, both calling the same runtime. They are building blocks for embedding rather than a deployment; a service that needs them wires them into its own HTTP stack. A REST caller names the document over the network, so it runs limited to `runtime.complexity` and `runtime.state`; a scheduler job’s path is the operator’s own, so it is trusted like `cof run ./file.yml` unless the job sets `trust_document=False`.

12.6 Observability

The run record *is* the observability. Three layers, from cheapest to most durable:

The shadow state file. `--out` at the end, `--live-state` as it happens, `--json` on stdout. Every effect’s value, every meta — adapter, model, `model_reason`, `prompt_sent`, tokens, timing, error, fallback chain, complexity score and band, decomposition record — and `runtime.effective_settings.sources` saying where every setting came from. A loop’s last is stored once, as a `{"$ref": "iter_<N>"}` to the pass it aliases ([Shadow state](#)). The TUI’s Runs view is a browser for exactly this file, and it links those references back.

Lifecycle hooks. Runtime plugins subscribe to `on_run_start`, `on_run_success`, `on_run_failure`, and the balanced per-effect pair `on_effect_start` / `on_effect_complete`. `on_effect_start` fires immediately *before* dispatch, carrying the node as it stands — the resolved model, the rendered prompt, the score — which is where a routing decision is observed at the moment it is made; `on_effect_complete` fires once value is final, including on failure. A use child’s effects fire inside their parent’s pair at namespaced paths. Plugins observe; they never change control flow, and a plugin that raises is recorded under `runtime.plugins.events` and isolated from the run.

Exporters. Thirty-odd runtime plugins ship in-tree: OpenTelemetry, Sentry, Datadog, Honeycomb, Prometheus, Loki, CloudWatch, and the persistence backends — [Tools and persistence](#) lists them. They are registered in config, not in documents.

12.7 Anti-patterns

Parsing prose output. `cof run` prints for humans on a terminal and JSON when piped. Scripts read `--json` or `--tail`; nothing else is stable.

Reaching into runtime.* keys that are not documented as public. [Stability](#) names the state paths you can rely on.

Driving the MCP loop one prompt at a time when pending_prompts has three. They are parallel branches; answer them all, in any order.

12.8 See also

- [Terminal UI](#) · [The Wizard](#) · [API Reference](#) · [Stability & Versioning](#).
- [.claude/commands/cof.md](#) — the MCP tool loop, as a slash command.

Chapter 13

Tools, adapters, and persistence

Three kinds of extension carry a run beyond the process, and all three are behind one small protocol each. **Tool plugins** reach outward from inside a document — a computation whose result is not a token stream. **Adapters** carry the conversation to a model provider. **Runtime plugins** are pure observers, and the durable ones — persistence backends — are the reason a run can be resumed, queried, and audited after the process is gone.

13.1 Tool effects

```
Tool ::= { type: 'tool', name: NAME, provider: PLUGIN_NAME,
  prompt?: TEMPLATE, model?: STRING,
  params?: MAP,                                - string values Mustache-rendered;
  ↪ wins over prompt/model
  params_json?: TEMPLATE,                      - rendered, parsed as JSON,
  ↪ deep-merged over params
  timeout_ms?: INT, on_error?: 'fail'|'skip'|'continue', description?: STRING }
```

A tool effect is a plugin call. Reading a file, fetching a URL, querying SQL, sending a Slack message, running ffmpeg — each fits the same monadic shape as a prompt, writes to the same deterministic path, and participates in control flow indistinguishably from one. An if can branch on a tool's result; a loop can iterate it; a prompt can read it.

```
- type: tool
  name: fetch_recipe
  provider: web_fetch
  params:
    url: "{{input.recipe_url}}"
  on_error: skip
```

provider names the plugin. params is the plugin's own vocabulary — every string value is a Mustache template — and for the two media plugins that predate params, prompt and model are top-level shorthand (params wins when both are given). The result lands at prime.<name>.value in whatever shape the plugin documents: a path, a parsed document, a list of records.

A value rendered into params is always a string, so a list from an earlier step arrives as its string form. When a tool needs a real array or object built at run time, write params_json: a template that renders to a JSON object, which is parsed and deep-merged over params (its keys win). A list or object from state renders into it as JSON:

```
- type: tool
  name: open_issue
  provider: mcp
  params:
    server: github
    tool: create_issue
    arguments: {owner: "{{input.owner}}", repo: "{{input.repo}}", title: "Tonight's menu"}
    params_json: '{"arguments": {"labels": {{{prime.menu.plan_courses.value}}}}'
```

Here `arguments.labels` is the course list itself, next to the three strings from `params`. A template that does not render to a JSON object fails the tool; it never runs with other parameters than the ones you wrote.

The division of labour is strict. Tools are for what a model *cannot* do — side effects and observations. Text work — summarising, extracting, classifying, writing, coding — is a prompt, and the pattern that joins them is **prompt-then-tool**: a json prompt produces the parameters, a tool executes them.

```
- type: prompt
  name: plan_poster
  prompt_type: json
  schema:
    type: object
    properties:
      image_prompt: {type: string}
      width: {type: number}
      required: [image_prompt]
    template: "Describe a poster for tonight's menu as an image prompt. Return ONLY JSON with
    ↪ image_prompt and width."

- type: tool
  name: render_poster
  provider: comfyui
  prompt: "{{prime.plan_poster.value.image_prompt}}"
  model: flux1-schnell-fp8.safetensors
  params:
    width: "{{prime.plan_poster.value.width}}"
    height: 768
    image_dir: ./output/posters
```

13.1.1 The built-in providers

Seventy-odd ship in-tree. By purpose:

Group	Providers
Time / utility	clock, math, regex, json, uuid, hash, base64, hex, validate_yaml
Filesystem / data	fs, csv, tar, zip, gzip, 7z
Internet	http, web_search, web_fetch, webhook, wikipedia, rss, weather
Browser	playwright, screenshot
Communication	email_smtp, slack, discord
Productivity SaaS	github, jira, linear, notion, gcalendar, gdrive
Storage	s3, surrealdb

Group	Providers
Audio / image / video	ffmpeg, comfyui, imagemagick, exiftool, ocr, yt_dlp, mediainfo
PDF / documents	pdf_extract, pdf_render, pandoc
Embeddings / RAG	embed, rerank, vector_search
Code / dev	git, gh, ripgrep, pytest, linter, docker, kubectl
Sandboxed execution	python_eval, shell
Text processing	awk, sed, diff_patch
Network	dns, whois, ping, traceroute, port_check
System	system_info, process_list, env_vars
Crypto / markup	gpg, xml, html_extract
MCP	mcp — any tool on any MCP server

`cof list --extensions` prints the exact compiled-in set. Plugins that need optional PyPI packages or external binaries lazy-import; `cof doctor` reports each one's readiness, and `pip install circuitry-cof[<plugin>]` ([playwright], [github], [embed], ...) pulls what a plugin needs. `docs/plugins/` documents the media plugins' parameters in full.

The plugins that wrap a command-line program (imagemagick, ffmpeg, git, pandoc, yt_dlp, and the other subprocess tools) search `PATH` for it by default. Config can name the exact executable and add to its environment — both machine-specific, so both stay out of the document:

```
{
  "runtime": {
    "plugins": {
      "imagemagick": {
        "binary": "~/opt/imagemagick-omp/bin/magick",
        "env": {"MAGICK_THREAD_LIMIT": "4"}
      }
    }
  }
}
```

`binary` is an absolute path (~ is expanded) and replaces the `PATH` search; `env` is merged over the inherited environment. A configured binary that does not exist fails the tool and preflight with a message that names the setting. Each tool node records the executable that actually ran in `meta.binary`. [Binary tool plugins](#) lists the plugins that read these settings.

Two are gated on purpose. `shell` runs a single binary from an allowlist that is deliberately tiny and read-only by default, with a per-effect `allowed_commands` override the author must write down; `python_eval` is likewise sandboxed. Shell metacharacters are rejected in `ffmpeg` paths. And the `enabled_tools` allowlist in config is the deployment-level gate: a document that references a tool outside it — itself or through a use child — fails validation before anything runs, and a tool that only appears at run time, in a rendered inline: child or a generated plan, is refused before it is built.

13.1.2 MCP servers as tool providers

The `mcp` provider is the client-side complement to the `MCP server` in [Surfaces](#): it calls tools on external Model Context Protocol servers. Servers are declared in config — named, like adapters, and referenced from YAML by name only, so credentials never enter a document and are redacted before any state is written:

```
{
  "runtime": {
    "plugins": {
      "mcp": {
        "servers": {
          "github": {"command": "npx", "args": ["-y",
            ↪ "@modelcontextprotocol/server-github"], "env":
            ↪ {"GITHUB_PERSONAL_ACCESS_TOKEN": "..."}},
          "internal": {"url": "https://mcp.example.com/mcp", "headers": {"Authorization":
            ↪ "Bearer ..."}}
        }
      }
    }
  }
}
```

```
- type: tool
  name: open_issue
  provider: mcp
  params:
    server: github
    tool: create_issue
    arguments:
      owner: "{{input.owner}}"
      repo: "{{input.repo}}"
      title: "Tonight's menu: {{prime.plate.value}}"
```

Transport is inferred from the shape (command → stdio, url → HTTP). The value is the tool's structured content when the server sends it; otherwise, under the default `parse: auto`, its text is parsed as JSON when it is a JSON object or array, and kept as text when not. `operation: list_tools` returns a server's catalogue — a way for an orchestration, or a reflector, to discover capabilities at run time.

13.2 Adapters

An adapter carries one call out of the process:

```
class Adapter(Protocol):
    name: str
    def generate(self, *, model: str, prompt: str, timeout_seconds: int = 120) ->
        ↪ GenerateResult: ...
    def check(self) -> CheckResult: ...           # preflight
```

`GenerateResult` is text, raw (the provider's response, verbatim), and optional `tokens_sent` / `tokens_received`. That is the whole contract, and twenty-nine adapters implement it: `ollama`, `openai`, `anthropic`, `gemini`, `mistral`, `cohere`, `groq`, `deepseek`, `xai`, `perplexity`, `together`, `fireworks`, `replicate`, `ai21`, `qwen-dashscope`, `nvidia-nim`, `huggingface-inference`, `watsonx`, `databricks`, `azure-openai`, `cloudflare-workers-ai`; the self-hosted servers `vllm`, `llamacpp`, `lmstudio`, `tgi`; the aggregators `openrouter`, `litellm`; the cyberdiner broker; and `host_claude`.

An orchestration written for one runs on another with no edits, because the document never named the adapter — [Configuration](#) did. Adapters declare an optional `list_models()` (what `cof list --models <adapter>` calls) and are registered in `ADAPTER_REGISTRY`; [Adapter Conformance](#) is the checklist for writing one, and the `adapter=` parameter of the SDK is how to use one without registering it.

13.3 Runtime plugins

A runtime plugin observes a run and changes nothing about its semantics:

```
class MyPlugin:
    name = "my-plugin"
    def on_run_start(self, *, state, context): ...
    def on_run_success(self, *, state, context): ...
    def on_run_failure(self, *, state, context, error): ...
    def on_effect_start(self, *, state, context, effect_path, effect_node): ... #
    ↪ optional
    def on_effect_complete(self, *, state, context, effect_path, effect_result): ... #
    ↪ optional
```

context carries run_id, orchestration_path, dry_run, validate_only, and runtime_config. The per-effect pair is balanced — an effect that fires one fires the other, failure included — and namespaced through use children. Plugins register in config ("plugins": ["my_package.plugins:make_plugin"], a module:attr that may be a zero-argument factory); a plugin that raises is recorded under runtime.plugins.events and never fails the run. The contract version is reported at runtime.plugins.contract_version.

Thirty-one ship in-tree, behind the one protocol:

Kind	Plugins
SQL persistence (one schema, seven dialects)	sqlite, postgres, mysql, mssql, cockroachdb, duckdb, clickhouse
Document / KV / object stores	mongodb, couchdb, firestore, dynamodb, elasticsearch, opensearch, surrealdb, redis, memcached, s3, gcs, azure_blob, r2
Append log	jsonl_file
Pub/sub	kafka, nats, rabbitmq
Observability exporters	opentelemetry, sentry, datadog, honeycomb, prometheus, loki, cloudwatch

The exporters turn the per-effect pair into spans, events, or metrics; the stores turn the run record into rows. None of them touch control flow, which is the point of the protocol: you can add Datadog to a deployment without re-validating a single orchestration.

13.4 Persistence: the durable side of the shadow state

runtime.persistence selects a backend that stores each run's final state and rehydrates it for the next:

```
{
  "runtime": {
    "persistence": {
      "enabled": true,
      "backend": "sqlite",
      "db_path": ".circuitry/runs.db"
    }
  }
}
```

backend	Required	Notes
jsonl-file	path	one record per run, appended; stdlib; stays greppable
sqlite	db_path (or path)	stdlib; table defaults to circuitry_runs
postgres	dsn	pip install psycopg[binary]; sslmode should be require or stronger
mongodb	uri	pip install circuitry-cof[mongodb]; database / collection

With a backend configured, a run loads the last persisted state for the orchestration before it starts (`runtime.persistence.loaded_from_persistence: true`) and saves its own at the end. A backend that cannot be reached fails the run with an actionable error and records `runtime.persistence.status`. Credential-bearing values — a DSN’s password, a Mongo URI’s `user:pass@` — are redacted everywhere state is serialized; only the driver sees them. A [profile](#) can select a backend per run, and `--out` remains what it always was: a one-shot dump of the final state to a file, orthogonal to and combinable with any backend.

[Postgres Persistence](#) has the production notes.

13.5 Anti-patterns

A tool for text work. Summarising is a prompt. A tool that “analyses” is a prompt wearing a provider:.

Hard-coding tool parameters the model should choose. Let a json prompt produce them; the tool reads `{{prime.<prompt>.value.<field>}}`.

Widening the shell allowlist by habit. Each `allowed_commands` entry is a sentence in your threat model.

A plugin that steers. Runtime plugins observe. A plugin that mutates prime mid-run has broken the one property everything else relies on.

Credentials in a document. Servers, tokens, DSNs — config, always. The run record redacts config; nothing redacts a template.

13.6 See also

- [Plugin Extension Guide](#) · [Adapter Conformance](#) · [Postgres Persistence](#).
- [docs/plugins/ffmpeg.md](#) · [docs/plugins/comfyui.md](#) · [docs/plugins/surrealdb.md](#) · [Binary tool plugins](#) · [Runtime Plugin Catalog](#).
- [Threat Model](#).

Chapter 14

The whole meal

Every part of the language, in one document. Two files: the dinner party, and the sauce sub-recipe it calls.

```
## make_sauce.yml – the child: a typed interface and one prompt
interface:
  inputs:
    base: {type: string, required: true, description: The dish the sauce accompanies.}
    style: {type: string, required: false}
  outputs:
    recipe: {type: string, path: prime.compose.value}

effects:
- type: prompt
  name: compose
  template: "Compose a {{input.style}} sauce for {{input.base}}, in three sentences."
```

```
## dinner_party.yml
## Inputs: occasion (required), diet, recipe_url. Output: prime.plate.value
interface:
  inputs:
    occasion: {type: string, required: true}
    diet: {type: string, required: false}
    recipe_url: {type: string, required: false}
  outputs:
    dinner: {type: string, path: prime.plate.value}

effects:
# tool – fetch inspiration; no internet, still dinner
- type: tool
  name: fetch_recipe
  provider: web_fetch
  params: {url: "{{input.recipe_url}}"}
  on_error: skip

# if – branch on the guest list; both branches write `main_course`
- type: if
  name: check_diet
  if: {mode: cel, expr: "state.input.diet == 'vegetarian'"}
  then:
    - type: prompt
      name: main_course
      template: "Suggest a vegetarian main course for {{input.occasion}}, in one
↪ sentence."
```

```

else:
  - type: prompt
    name: main_course
    template: "Suggest a main course for {{input.occasion}}, in one sentence."

# use – the sauce sub-recipe; its interface supplies the output mapping
- type: use
  name: sauce
  path: ./make_sauce.yml
  inputs: {base: "{{prime.check_diet.main_course.value}}", style: pan}

# dynamic – the menu, planned in a named scope; typed output feeds the loop below
- type: dynamic
  name: menu
  flow: chain
  effects:
    - type: prompt
      name: plan_courses
      prompt_type: array
      schema: {type: array, items: {type: string}}
      template: |
        List three courses around {{prime.check_diet.main_course.value}}.
        Inspiration, if any: {{{prime.fetch_recipe.value}}}
        Return ONLY a JSON array of strings.
    - type: prompt
      name: wine
      template: "Pick one wine for this menu: {{prime.menu.plan_courses.value}}"

# loop (each) – cook every course in parallel, collect the steps
- type: loop
  name: courses
  flow: tree
  each: {in: prime.menu.plan_courses.value, as: course}
  collect: cook
  body:
    - type: prompt
      name: cook
      template: "Write the cooking steps for: {{course}}"

# loop (while) – season, taste, adjust; the model is the palate
- type: loop
  name: season
  while:
    mode: model
    template: "Taste this. Does the seasoning need adjusting?\n\n{{prime.adjust.value}}"
  min_iterations: 1      # always taste at least once, so `last` exists below
  max_iterations: 3
  body:
    - type: prompt
      name: adjust
      template: "Adjust the seasoning of {{prime.check_diet.main_course.value}} and
↪ describe the dish now."

# reflector – plan the final hour of service live; the planner reads only its own
↪ template
- type: reflector
  name: replan_service
  max_effects: 4
  effects:

```

```

- type: prompt
  name: propose_steps
  template: |
    The courses are cooked and the main course is seasoned. The guests sit down in
    ↪ one hour.
    Plan the final hour of service. Output must follow the OUTPUT CONTRACT exactly.

# plate – one prompt reads across everything the run produced
- type: prompt
  name: plate
  template: |
    Write tonight's menu card.
    Main: {{{prime.season.last.adjust.value}}}
    Sauce: {{{prime.sauce.value.recipe}}}
    Wine: {{{prime.menu.wine.value}}}
    Courses: {{{prime.courses.collected.value}}}

cof check dinner_party.yml
cof run dinner_party.yml -e occasion=anniversary -e diet=vegetarian --live-state
↪ dinner.live.json

```

14.1 Reading the run

dinner.live.json is the state graph filling in as dinner cooks. When it is done, the shape is exactly what the names dictate:

input.occasion	"anniversary"
input.diet	"vegetarian"
prime.fetch_recipe.value	null – skipped; meta.error says why
prime.check_diet.value.branch	"then"
prime.check_diet.main_course.value	the main course
prime.sauce.value.recipe	the sauce, via the child's interface
prime.menu.plan_courses.value	["...", "...", "..."]
prime.menu.wine.value	
prime.courses.iter_0.cook.value ... iter_2	one pass per course, cooked in parallel
prime.courses.collected.value	the three sets of steps, in menu order
prime.courses.value.termination.reason	"collection_exhausted"
prime.season.value.termination.reason	"condition_false", or
↪ "max_iterations_reached" if never satisfied	
prime.season.iter_0.adjust.value ...	each tasting pass
prime.season.last	{"\$ref": "iter_<N>"} – the final pass,
↪ stored once	
prime.replan_service.generated.iter_0...	whatever the planner decided to run
prime.plate.value	the menu card
runtime.effective_settings.sources	where every setting came from

Every effect's meta sits beside its value — which model, why, the rendered prompt, tokens, timing, any fallback that answered. The file stores season's final pass once and points last at it; the run itself read prime.season.last.adjust.value as a value when plate rendered, and --state or the TUI's Runs view link the reference back when they load the file.

14.2 Turning on the complexity layer

Nothing in the document changes. In config:

```
{
  "runtime": {
    "complexity": {
      "scoring": {"enabled": true},
      "routing": {
        "enabled": true,
        "bands": [
          {"name": "light", "max": 20, "model": "phi3:mini"},
          {"name": "mid", "max": 40, "model": "qwen2.5:7b-instruct"},
          {"name": "heavy", "max": 60, "model": "gpt-oss:20b"}
        ]
      },
      "decomposition": {"enabled": true, "threshold": 45, "max_chunks": 5, "max_depth": 1}
    }
  }
}
```

```
cof score dinner_party.yml
cof run dinner_party.yml -e occasion=anniversary --explain-routing --decompose-out ./plans
```

The cheap prompts — wine, cook — route to the small model; plate, reading across the whole run, scores higher and routes up; and if `plan_courses` ever grows past the threshold, it decomposes into courses of its own and merges back at `prime.menu.plan_courses.value`, where the loop reads it unchanged.

14.3 What the meal demonstrates

Chapter	In the document
Prompt	typed output (<code>plan_courses</code> → array), triple-stache for prose
Dynamic	menu chain; courses fans out with <code>flow: tree</code>
Shadow state	every path derived from names; <code>input. /</code> <code>prime. / runtime.</code>
Configuration	no <code>adapter:</code> or <code>model:</code> anywhere in the document
Errors	<code>on_error:</code> skip on the optional fetch; a template written for an empty value
If	<code>check_diet</code> , same name in both branches
Loop	each with <code>collect</code> ; while with the model as sensor; last after the loop
Reflector	a bounded planner whose template states the situation
Composition	<code>make_sauce.yml</code> with an interface; outputs auto-generated
Complexity, Decomposition	switched on in config, invisible to the document
Surfaces	<code>--live-state</code> , <code>--explain-routing</code> , <code>--decompose-out</code>
Tools	<code>web_fetch</code> as an effect like any other

An orchestration is a declared control mechanism which, through reflectors and decomposition, lays its own plans. Dinner is served.

Grammar

A.1 Terminals

A.2 Document

79

```

InputDecl    ::= { type?: 'string'|'number'|'boolean'|'array'|'object',
                    required?: BOOL,           - default false
                    description?: STRING }
OutputDecl   ::= { path: STATE_PATH, type?: STRING, description?: STRING }
                | STATE_PATH                  - shorthand for {path: ...}

```

A.3 Effect — discriminated by type, seven productions

Effect ::= Prompt | Dynamic | If | Loop | Tool | Use | Reflector

A.3.1 Prompt — one model call → prime.<name>.value

```

Prompt ::= { type: 'prompt', name: NAME,
              template: TEMPLATE @ messages: Message+,
              prompt_type?: 'text'|'json'|'object'|'array'|'number'|'boolean'|'tool', -
              ↪ default text
              schema: JSONSCHEMA,           - REQUIRED iff prompt_type ∈
              ↪ {json,object,array}
              model?: STRING, provider?: STRING, provider_fallbacks?: STRING*,
              params?: MAP, inputs?: MAP,    - inputs: prompt-local template
              ↪ vars
              assets?: Asset*, retries?: Retry,
              timeout_ms?: INT, deterministic?: BOOL,
              on_error?: OnError, description?: STRING }

```

Message ::= { role: 'system'|'user'|'assistant'|'tool', content: STRING }

Asset ::= { kind: STRING, ref: STRING }

Retry ::= { max_attempts?: INT≥1, backoff_ms?: INT }

A.3.2 Dynamic — named container → prime.<name>.<child>.value

```

Dynamic ::= { type: 'dynamic', name: NAME, effects: Effect+,
              flow?: Flow, max_concurrency?: INT≥1, - tree only
              stop_on_error?: BOOL, on_error?: OnError,
              labels?: MAP, description?: STRING }

```

A.3.3 If — one branch runs

```

If ::= { type: 'if', if: Condition, then: Effect*,
         else?: Effect*, name?: NAME,           - named ⇒ nested paths + decision
         ↪ meta;                               unnamed ⇒ branch effects merge
                                                ↪ into parent scope
         threshold?: NUMBERT[0,1],             - model-mode confidence, default
         ↪ 0.5
         on_error?: OnError, labels?: MAP, description?: STRING }

Condition ::= { mode: 'model', template: TEMPLATE } - LLM answers yes/no
              | { mode: 'cel', expr: CEL,           - deterministic
                strict?: BOOL }                   - default false; true ⇒ unset path
              ↪ is an error

```

A.3.4 Loop — each xor while

```

Loop ::= { type: 'loop', body: Effect+,
           each: { in: STATE_PATH | BINDING_PATH, - resolves to an array

```

as?: NAME,	- default 'item'
truncate?: BOOL }	- default false; see
↪ max_iterations	
⊕ while: Condition,	- checked before each pass;
↪ always sequential	
name?: NAME,	- named ⇒ iter_<N>/last/collected
↪ nodes;	
	unnamed ⇒ body overwrites at
	↪ stable paths
collect?: NAME,	- a body step; needs a named loop
flow?: Flow, max_concurrency?: INT≥1,	- each-loops only
max_iterations?: INT≥1,	- no default (uncapped); each: a
↪ longer collection	
	fails at start unless
	↪ each.truncate
min_iterations?: INT,	
on_error?: OnErrorLoop, labels?: MAP, description?: STRING }	

Read grammar (post-loop unless noted): prime.<step>.value (this pass, body/while only)
 · prime.<loop>.iter_<N>.<step>.value · prime.<loop>.last.<step>.value (last *completed* pass) · prime.<loop>.collected.value. In a saved shadow state last is {"\$ref": "iter_<N>"}, linked back on load. Termination (prime.<loop>.value.termination.reason):
 collection_exhausted · condition_false · max_iterations_reached (+ unvisited when truncated) · collection_unresolved · condition_error · error.

A.3.5 Tool — deterministic plugin call → prime.<name>.value

```
Tool ::= { type: 'tool', name: NAME, provider: PLUGIN_NAME,
  prompt?: TEMPLATE, model?: STRING,
  params?: MAP,                                - string values Mustache-rendered;
    ↪ wins over prompt/model
  params_json?: TEMPLATE,                      - renders to a JSON object;
    ↪ deep-merged over params
  timeout_ms?: INT, on_error?: OnError, description?: STRING }
```

A.3.6 Use — child orchestration in isolation

```
Use ::= { type: 'use', name: NAME,
  ref: LIBRARY_REF ⊕ path: FILE_PATH ⊕ inline: TEMPLATE, - inline renders to
    ↪ YAML at runtime
  validate?: BOOL,                                - default true; schema-gates
    ↪ inline YAML
  inputs?: { NAME: value|TEMPLATE|REF ... },        - become the child's input.*;
    ↪ TEMPLATE arrives as text
  outputs?: { NAME: OutputDecl ... },              - omitted ⇒ full child namespace
    ↪ exposed
  on_error?: OnError, description?: STRING }
```

LIBRARY_REF ::= '<category>/<name>' | '<source>:<category>/<name>'

A.3.7 Reflector — plans its own effects

```
Reflector ::= { type: 'reflector', name: NAME, effects: Effect+, - base template per
  ↪ cycle
  plan_from_step?: NAME,                        - default propose_steps
  max_iterations?: INT≥1,                       - default 1
  generated_key?: STRING,                       - default generated
  stop_on_done?: BOOL,                         - default true
```

```

max_effects?: INT $\geq$ 1,           – default 8
prime_template?: STRING, flow?: Flow, description?: STRING }

```

A.4 Static semantics (beyond shape)

1. Sibling names unique; names are path segments, never paths; never type-keywords (the validator rejects duplicates and warns on type-keyword names).
2. `each.in`, CEL paths, use input references, and template refs must root at a namespace — or, inside a loop, at an enclosing loop’s binding (`course.steps` for `each.in` and references, `state.course / state.iter.index` in CEL); a declared `interface.inputs` name is referenced only as `{{input.<name>}}` / `state.input.<name>` — anything else is a compile error. Every CEL expression must parse.
3. `schema required` $\Rightarrow$ `structured prompt_type`; `template required` $\Rightarrow$ `mode: model`; `expr required` $\Rightarrow$ `mode: cel`.
4. `collect` must name a body step, and needs a named loop to have anywhere to write — on an unnamed loop it validates and silently aggregates nothing. Chain-order reads only; tree siblings can’t read each other.
5. `use cycle guard`: an orchestration cannot (transitively) use itself.
6. `runtime.*` in a document merges over `config` (orch wins) — the complexity block rides this.

A.5 Cybernetic overlay (config grammar, runtime.complexity)

```

Complexity ::= { scoring?:      { enabled?: BOOL, weights?: MAP, keywords?: MAP },
                  routing?:     { enabled?: BOOL, respect_explicit?: BOOL,
                                bands?: Band+ },           – ordered; final band omits max
                                ↪ (catch-all, required)
                  decomposition?: { enabled?: BOOL, threshold?: NUMBER, max_chunks?: INT,
                                max_depth?: INT, on_failure?: 'route_up'|'fail' } }
Band         ::= { name: STRING, max?: NUMBER, model: STRING } – max inclusive; models are
↪ opaque strings

```